
Logic and Computer Design Fundamentals

Chapter 6 – Registers and Register Transfers

Part 3 – Control of Register Transfers

Ming Cai

cm@zju.edu.cn

College of Computer Science and Technology
Zhejiang University

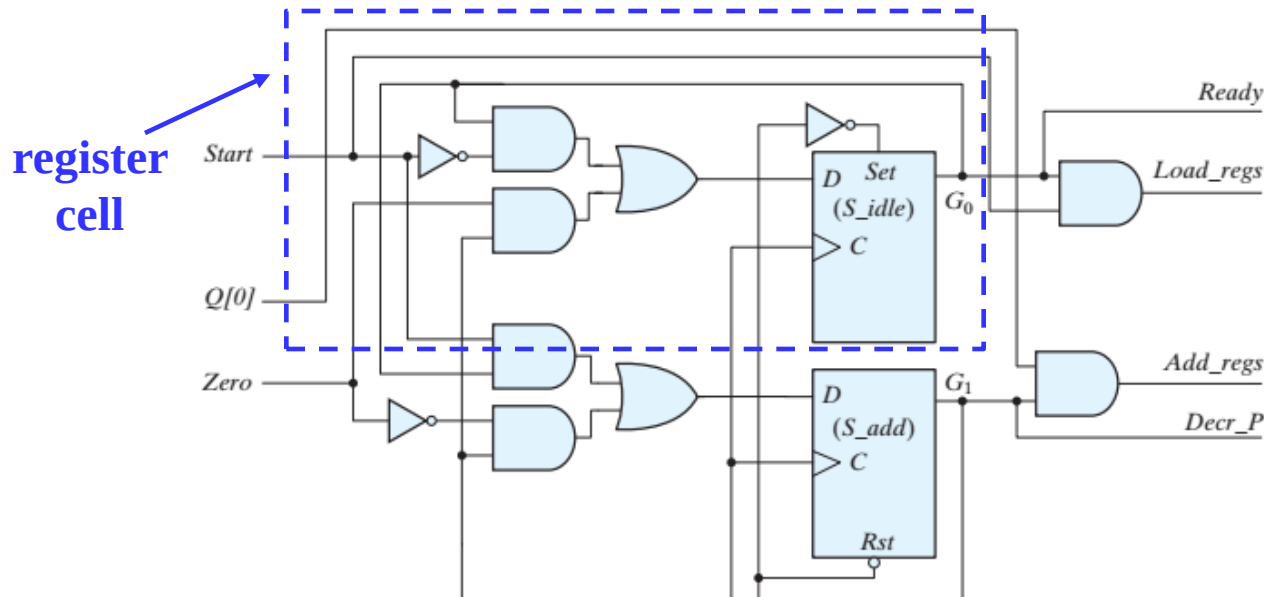
Overview

- **Part 1 – Registers, Microoperations and Implementations**
- **Part 2 – Counters, register cells, buses, & serial operations**
 - **Microoperations on single register (continued)**
 - **Counters**
 - **Serial transfers and microoperations**
 - **Register cell design**
- **Part 3 – Control of Register Transfers**

Iterative Sequential Circuits

- **In combinational circuit design, we use iterative array to implement a complex combinational circuit (e.g., ripple carry adder).**
- **Similarly, the idea of iterative array can also considerably simplify the design process of sequential logic circuits.**
- **We can connect iterative combinational circuits to flip-flops to form sequential circuits.**

Register Cell



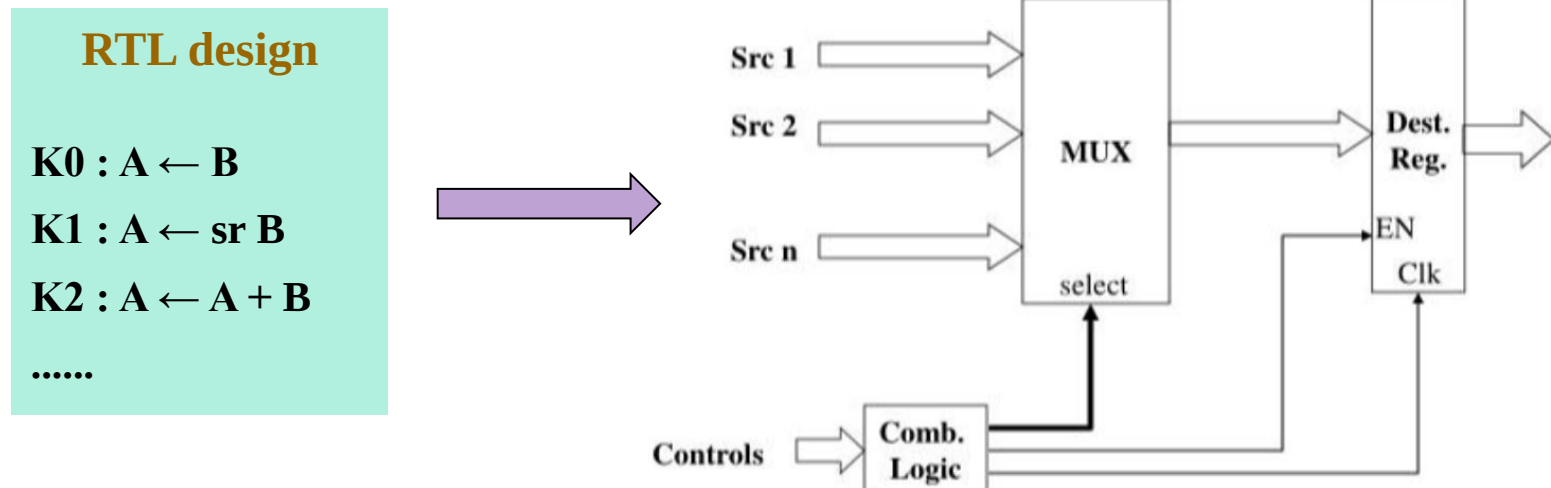
- A single-bit cell of **an iterative combinational circuit**, connected to **a flip-flop** that provides the output, forms a two-state sequential circuit called a **register cell**.
- We can design an n-bit register with one or more associated microoperations by designing a register cell and making n copies of it.

Register Cell Design

- Assume that a register consists of identical cells.
- Then the register design can be approached as follows:
 - Design representative cell for the register
 - Connect copies of the cell together to form the register
 - Applying appropriate “**boundary conditions**” to cells that need to be different and contract if appropriate
- Register cell design is the first step of the above process.

Register Cell Design (continued)

- Two design approaches for register cell design
 - **RTL-based Design**——**Multiplexer Approach**
 - Using multiplexers for selection to enable multiple microoperations on a register cell



- **Sequential Circuit Design**
 - Derive state diagram and state table for the circuit

Register Cell Specification

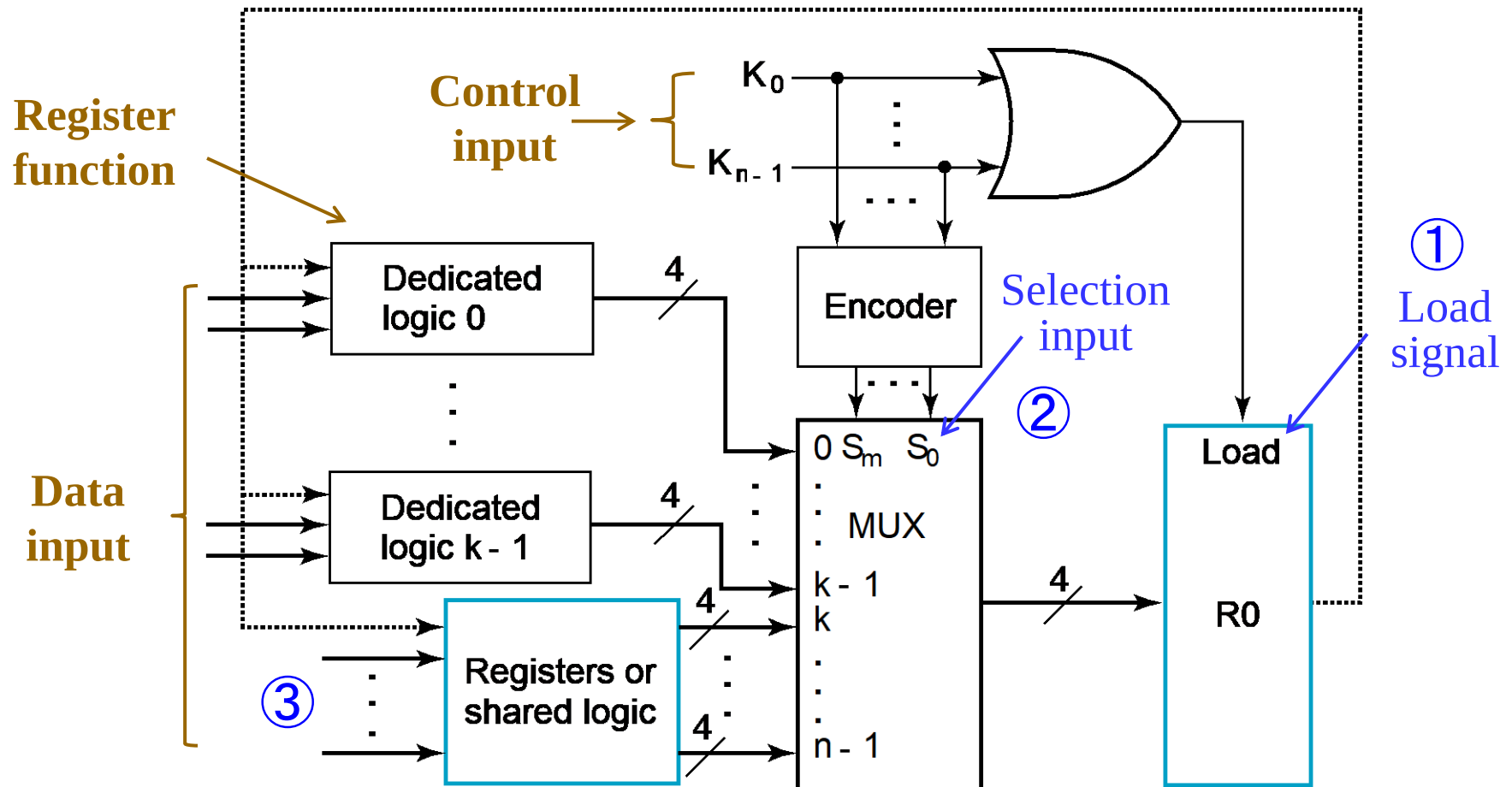
- According to register transfer operations, the specification of a register cell includes:
 - Register functions
 - Control input combinations to the register
 - Example 1: Not encoded
 - Control inputs: Load, Shift, Add
 - At most, one of Load, Shift, Add is 1 for any clock cycle (0,0,0), (1,0,0), (0,1,0), (0,0,1)
 - Example 2: Encoded
 - Control inputs: S1, S0
 - All possible binary combinations on S1,S0 (00,01,10,11)
 - Data inputs to the register

Register Cell Specification (continued)

- **Register functions** are typically specified as register transfers.
- **Register cell specification example:**
 - **Register functions:**
 - $K_0 : A \leftarrow B$
 - $K_1 : A \leftarrow \text{sr } B$
 - $K_2 : A \leftarrow A + B$
 - **Control inputs:** K_0, K_1, K_2
 - **A hold state:** if all control inputs are 0, hold the current register state
 - **Data inputs:** $B, \text{sr } B, A + B, A$

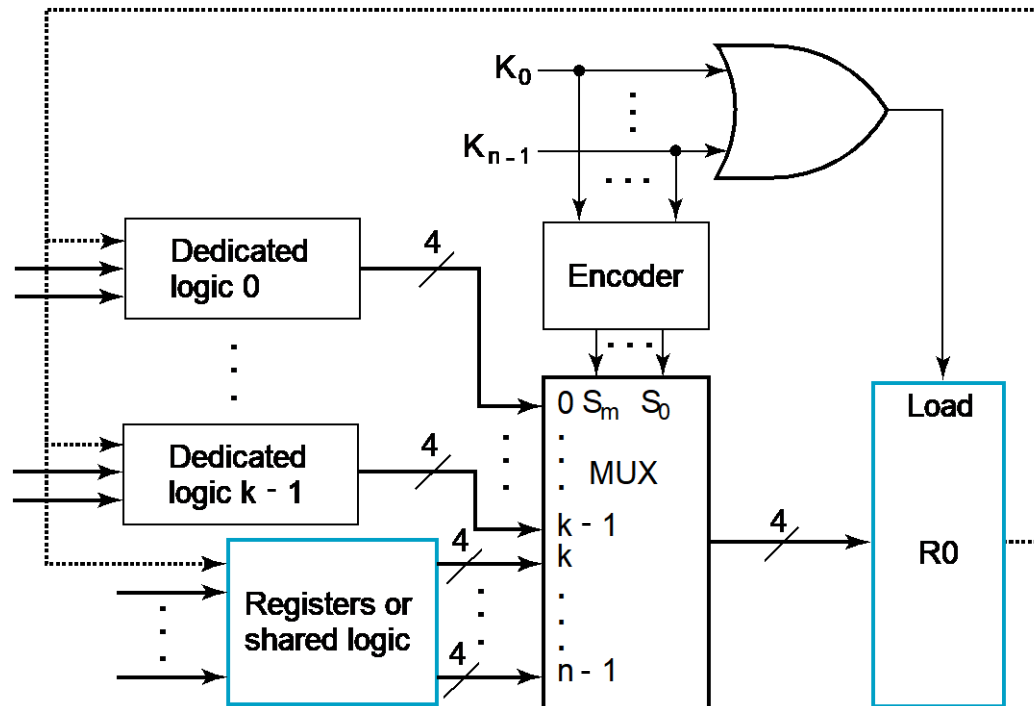
Multiplexer Approach

- Uses an n-input multiplexer with a variety of transfer sources and functions



Multiplexer Approach

- To produce the Load enable by OR of control signals $K_0, K_1, \dots, K_{n-1}$ (assumes no load for 00...0)
- To select sources and/or transfer functions:
 - Multiplexer + Encoder (shown): control inputs need to be encoded
 - $n \times 2$ AND-OR: using control inputs without encoding



Example 1: Register Cell Design

- **Register A (n-bits) Specification:**
 - **Register transfers:**
 - **CY:** $A \leftarrow B \oplus A$
 - **CX:** $A \leftarrow B \vee A$
 - **Hold state:** (0,0)
 - **Control inputs:** (CY, CX)
 - **Control input combinations** (0,0), (0,1) (1,0)
 - **Data inputs:** $B \oplus A$, $B \vee A$, A
- **Two design approaches**
 - **Multiplexer Approach**
 - **Sequential Circuit Design Approach**

Example 1: Register Cell Design (continued)

- **Multiplexer Approach**
- **Control inputs: (CY, CX)**
- **Load signal for register**

$$\text{Load} = \text{CX} + \text{CY}$$

- Since all control combinations appear encoded (0,0), (0,1), (1,0) can use multiplexer without an encoder:

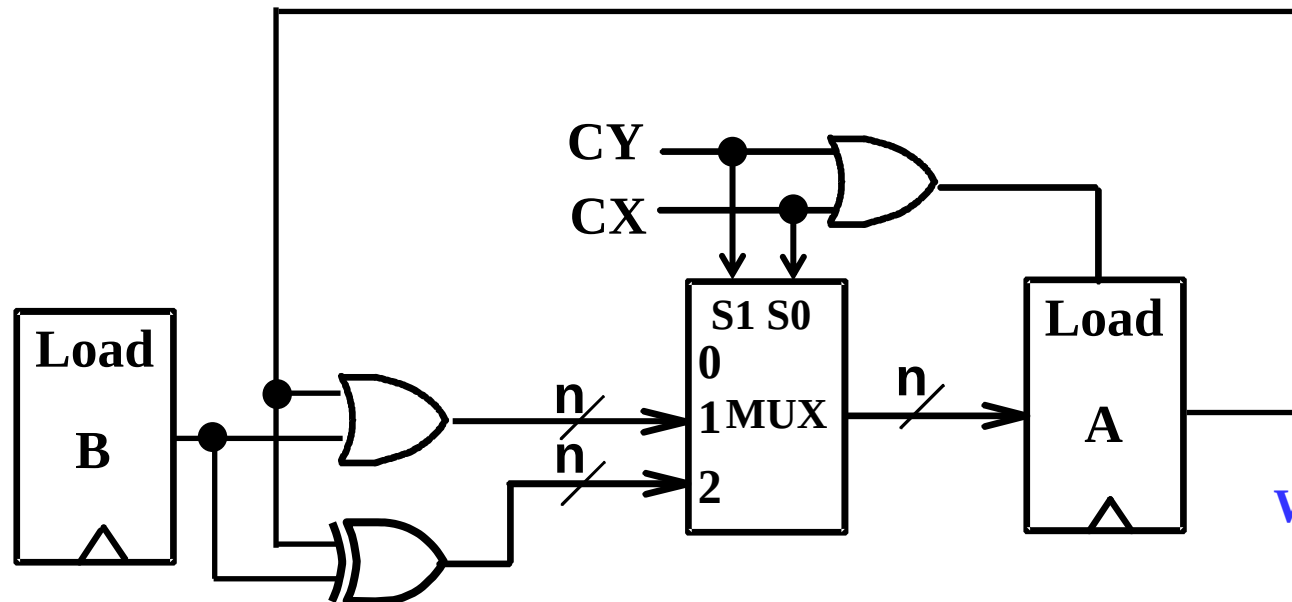
$$\begin{array}{l} \text{Selection inputs} \\ \text{for MUX} \end{array} \left\{ \begin{array}{l} S1 = \text{CY} \\ S0 = \text{CX} \end{array} \right.$$

$$\begin{array}{l} \text{Data inputs} \\ \text{for register} \end{array} \left\{ \begin{array}{ll} \text{Input}_1 = B_i \vee A_i & (\text{CY}, \text{CX}) = (0,1) \\ \text{Input}_2 = B_i \oplus A_i & (\text{CY}, \text{CX}) = (1,0) \\ \text{Input}_3 = A_i & (\text{CY}, \text{CX}) = (0,0) \end{array} \right.$$

Example 1: Register Cell Design (continued)

■ Register A (n-bits) Specification:

- **CY** : $A \leftarrow B \oplus A$
- **CX** : $A \leftarrow B \vee A$



Where is data input from register A?

The gate input cost with NOTs (GN) per cell

- Per cell: 19
- Shared decoder logic: 8

Sequential Circuit Design Approach

- **Find a state diagram or state table**
 - **Note that there are only two states with the state assignment equal to the register cell output value**
- **Use the design procedure in Chapter 4 to complete the cell design**
- **For optimization:**
 - **Use K-maps for up to 4 to 6 variables**
 - **Otherwise, use computer-aided or manual optimization**

Sequential Circuit Design Approach (continued)

- Register cell specification:

- Register functions:

- CY: $A \leftarrow B \oplus A$

- CX: $A \leftarrow B \vee A$

- Hold state: (0,0)

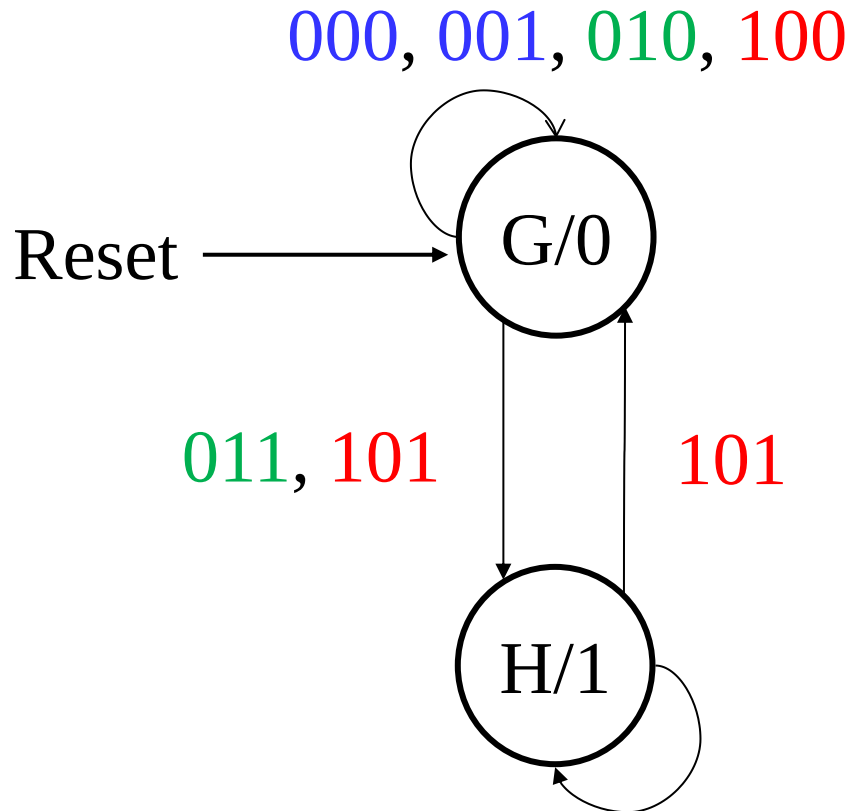
- Input: CY, CX, B

- Stored value: A

- Output: $Z = \begin{cases} A = B \oplus A & \text{if } (CY, CX) = 10 \\ A = B \vee A & \text{if } (CY, CX) = 01 \\ A = A & \text{if } (CY, CX) = 00 \end{cases} \text{ (Moore Model)}$

Example 1: Register Cell Design (continued)

■ Complete the state diagram:



- **Input:** CY, CX, B
- **State:**
 - **G:** A = 0
 - **H:** A = 1
- **Output:** Z
 - Z = state
- **Register functions:**
 - **CY:** $A \leftarrow B \oplus A$
 - **CX:** $A \leftarrow B \vee A$
 - **Hold state:** (0,0)

Moore model is assumed.

Example 1: Register Cell Design (continued)

■ State Table:

current state A	next state A(t+1) / D					
	Hold		$A \vee B$		$A \oplus B$	
	CY = 0 CX = 0 B = 0	CY = 0 CX = 0 B = 1	CY = 0 CX = 1 B = 0	CY = 0 CX = 1 B = 1	CY = 1 CX = 0 B = 0	CY = 1 CX = 0 B = 1
G (0)	0	0	0	1	0	1
H (1)	1	1	1	1	1	0

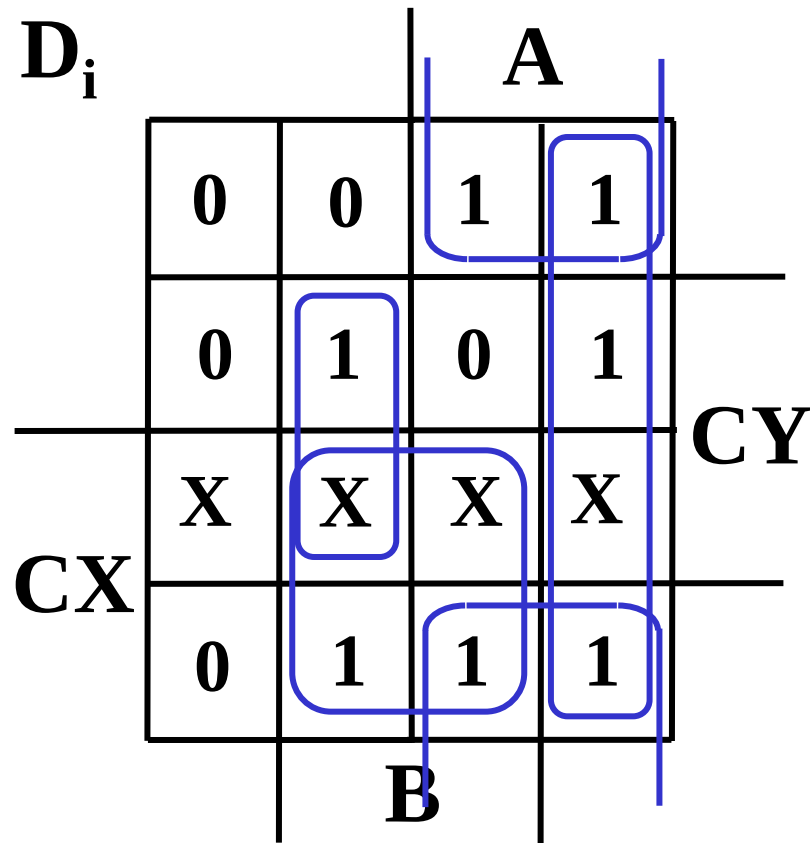
input

- Four variables give a total of 16 state table entries
- By using:
 - Combinations of variable names and values
 - Don't care conditions (for CX = CY = 1)

only 12 entries are required to represent the 16 entries

Example 1: Register Cell Design (continued)

- K-map - Use variable ordering CX, CY, A, B and assume a D flip-flop
- Input equation:



Example 1: Register Cell Design (continued)

- The resulting SOP equation:

$$D_i = CX B + CY \bar{A} B + A \bar{B} + \bar{C} \bar{Y} A$$

- Using factoring and DeMorgan's law:

$$D_i = CX B + \bar{A} (CY B) + A(\overline{CY B})$$

$$D_i = CX B + A_i \oplus (CY B)$$

The gate input cost per cell = $8 + 1 + 2 + 3 = 14$

- The gate input cost per cell for the multiplexer approach is:

Per cell: 19

Shared decoder logic: 8

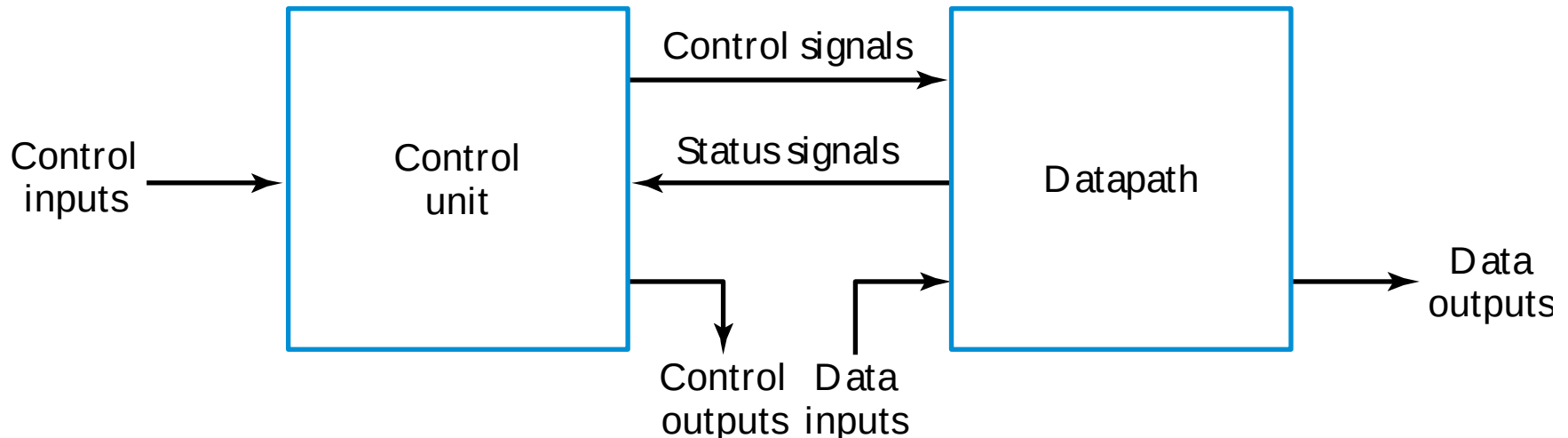
- Cost gain by sequential design > 5 per cell
- Also, no Load enable on the flip-flop makes it cost less

Overview

- **Part 1 – Registers, Microoperations and Implementations**
- **Part 2 – Counters, Register Cells, Buses, & Serial Operations**
- **Part 3 – Control of Register Transfers**
 - Introduction to register transfer systems
 - Register transfer system design procedure
 - A design example
 - Microprogrammed control

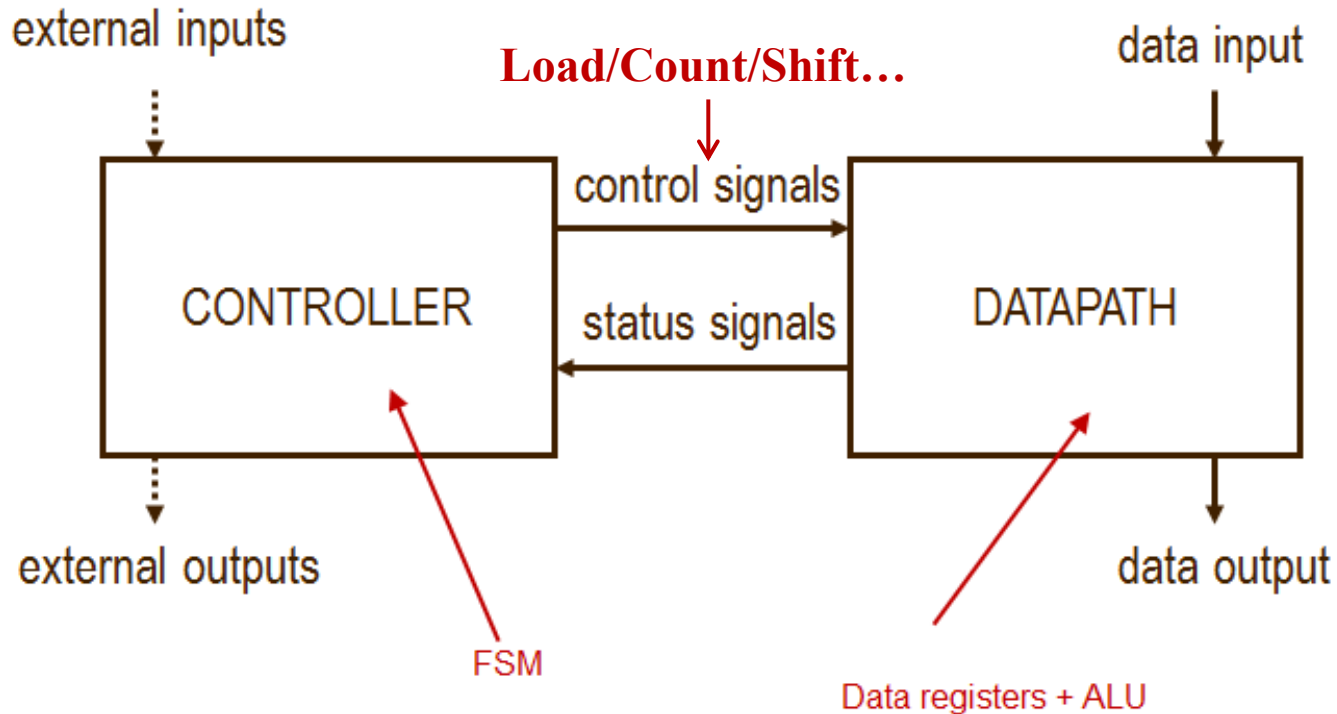
Introduction: Digital System Design Method

- **Three essential elements**
 - **Set of registers:** mostly in Datapath with some in Control Unit
 - **Basic operation (microoperation):** Register transfers performed on registers
 - **Control:** that supervises the sequencing of the register transfers



Datapath and Control Signals

▪ Datapath and Control Unit



- **Datapath** is a set of functional units (e.g., ALU, MUX), registers and buses that perform register transfer and micro-operations.
- **Control unit** generates the control signals to tell the datapath what to do.

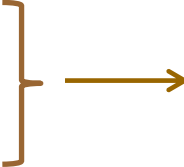
An Analogy——Bubble Sort

```
void bubble_sort(int arr[], int n) {  
    for (int i = 0; i < n - 1; i++) {  
        for (int j = 0; j < n - i - 1; j++) {  
            if (arr[j] > arr[j + 1]) {  
                int temp = arr[j];  
                arr[j] = arr[j + 1];  
                arr[j + 1] = temp;  
            }  
        }  
    }  
}
```

Control flow



Data flow



The Bubble Sort algorithm written in C language

An Analogy——Bubble Sort (continued)

bubble_sort:

```
push    rbp
mov     rbp, rsp
mov     QWORD PTR [rbp-24], rdi
mov     DWORD PTR [rbp-28], esi
mov     DWORD PTR [rbp-4], 0
jmp     .L36
```

.L40:

```
mov     DWORD PTR [rbp-8], 0
jmp     .L37
```

.L39:

```
mov     eax, DWORD PTR [rbp-8]
cdqe
lea     rdx, [0+rax*4]
mov     rax, QWORD PTR [rbp-24]
add     rax, rdx
mov     edx, DWORD PTR [rax]
mov     eax, DWORD PTR [rbp-8]
cdqe
add     rax, 1
lea     rcx, [0+rax*4]
mov     rax, QWORD PTR [rbp-24]
add     rax, rcx
mov     eax, DWORD PTR [rax]
cmp     edx, eax
jle     .L38
```

Control
flow

```
mov     eax, DWORD PTR [rbp-8]
cdqe
lea     rdx, [0+rax*4]
mov     rax, QWORD PTR [rbp-24]
add     rax, rdx
mov     eax, DWORD PTR [rax]
mov     DWORD PTR [rbp-12], eax
mov     eax, DWORD PTR [rbp-8]
cdqe
lea     rdx, [0+rax*4]
mov     rax, QWORD PTR [rbp-24]
add     rax, rdx
mov     edx, DWORD PTR [rbp-8]
movsx   rdx, edx
add     rdx, 1
lea     rcx, [0+rdx*4]
mov     rdx, QWORD PTR [rbp-24]
add     rdx, rcx
mov     eax, DWORD PTR [rax]
mov     DWORD PTR [rdx], eax
mov     eax, DWORD PTR [rbp-8]
cdqe
lea     rdx, [0+rax*4]
mov     rax, QWORD PTR [rbp-24]
add     rdx, rax
mov     eax, DWORD PTR [rbp-12]
mov     DWORD PTR [rdx], eax
```

Data flow

The Bubble Sort algorithm written in x86 assembly language

Register Transfer System Design Procedure (Hierarchical Design)

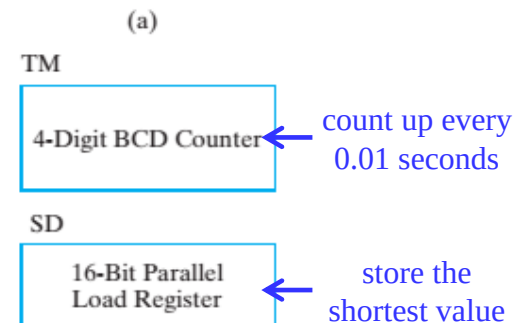
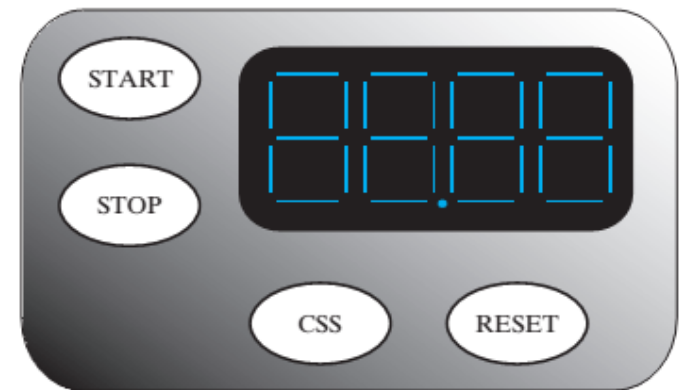
- Write a detailed system **specification**.
- Determine all data, control and status **input signals**, all data, control and status **output signals**, and **registers** of the datapath and control unit.
- Find a **state machine diagram** for the system including **register transfers** for the datapath and control unit as outputs.
- **Determine all internal control and status signals.**
Use these signals to separate output conditions and actions, including register transfers, from the state machine diagram flow and represent them in tabular form.

Register Transfer System Design Procedure (Hierarchical Design) (continued)

- **Draw a block diagram of the datapath** including all control and status inputs and outputs. Draw a block diagram of the control if it includes register transfer hardware.
- **Design any specialized register transfer logic** as needed for the datapath and the control.
- **Design the control unit logic.**
- **Verify the correct operation of the combined datapath and control unit.** If verification fails, debug the system and verify the changed system.

Design Example – DASHWATCH - Specs

- Stop Watch for “dash” runners
- Times intervals to at most 99.99 secs
- **Inputs:** START, STOP, CSS, RESET
 - The **START** button resets a timer to 0 and then starts the timer.
 - The **STOP** button stops the timer and the latest dash time is displayed.
 - The **CSS** button compares, stores and displays the minimum dash value.
 - The **RESET** button initializes the dash value to 10011001.10011001 (i.e., BCD equivalent of 99.99).
- **Output:** 4 digit BCD LCD with decimal point
- **Registers**
 - **4-digit BCD Counter (TM):** counts up every 0.01 seconds
 - **16-bit Parallel Load Register (SD):** stores the shortest dash value



DASHWATCH Inputs, Outputs, and Registers

	Symbol	Function	Type
Button	START	Initialize timer to 0 and start timer	Control input
	STOP	Stop timer and display timer	Control input
	CSS	Compare, store, and display shortest dash time	Control input
	RESET	Set shortest value to 10011001	Control input
LCD	B ₁	Digit 1 data vector a, b, c, d, e, f, g to display	Data output vector
	B ₀	Digit 0 data vector a, b, c, d, e, f, g to display	Data output vector
	DP	Decimal point to display (= 1)	Data output
	B ₋₁	Digit -1 data vector a, b, c, d, e, f, g to display	Data output vector
	B ₋₂	Digit -2 data vector a, b, c, d, e, f, g to display	Data output vector
	B	The 29-bit display input vector (B ₁ , B ₀ , DP, B ₋₁ , B ₋₂)	Data output vector
Register	TM	4-Digit BCD counter	16-Bit register
	SD	Parallel load register	16-Bit register

DASHWATCH State Machine Diagram with Register Transfer Outputs

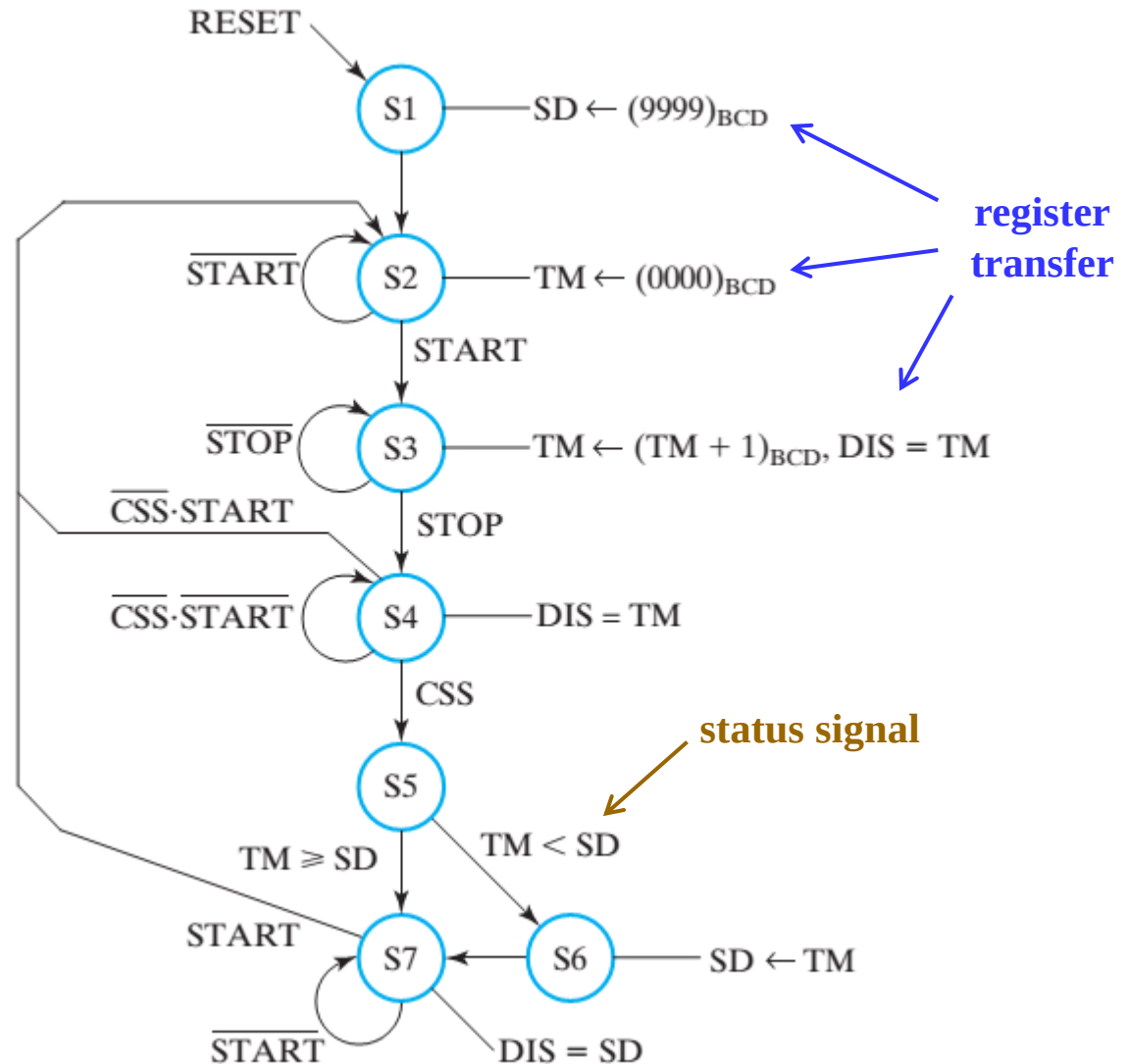
Which state machine
diagram is it?
Mealy or Moore?

Control signals
(external inputs)

- RESET
- START
- STOP
- CSS

Status signal
(from datapath)

- Compare TM and SD



State Machine Diagram Design

- Specify only Moore outputs (no particular reason)
- S1: Reset state - in this state, initialize SD to 1001100110011001 (99.99), the maximum possible dash time.
- S2: Because of Moore output spec, S1 cannot be used for this state since SD is not to be initialized again to 99.99 after having passed through states S4 or S7. TM is initialized to (0000)_{BCD} for next dash.
- S3: State during dash. Entered with START and exited with STOP. While in state, 1 (0.01 seconds) is added to TM for each clock pulse. (Clock frequency is 100 Hz), and DIS shows TM value.
- S4: Decision state whether to Compare, Store, and display Shortest dash time, or to continue to display TM. Also START begins new dash.
- S5: State for comparison of TM to SD.
- S6: State for loading TM into SD if TM is smaller.
- S7: State for START to begin new dash and display of SD as shortest dash time.

DASHWATCH Output Control/Status Table

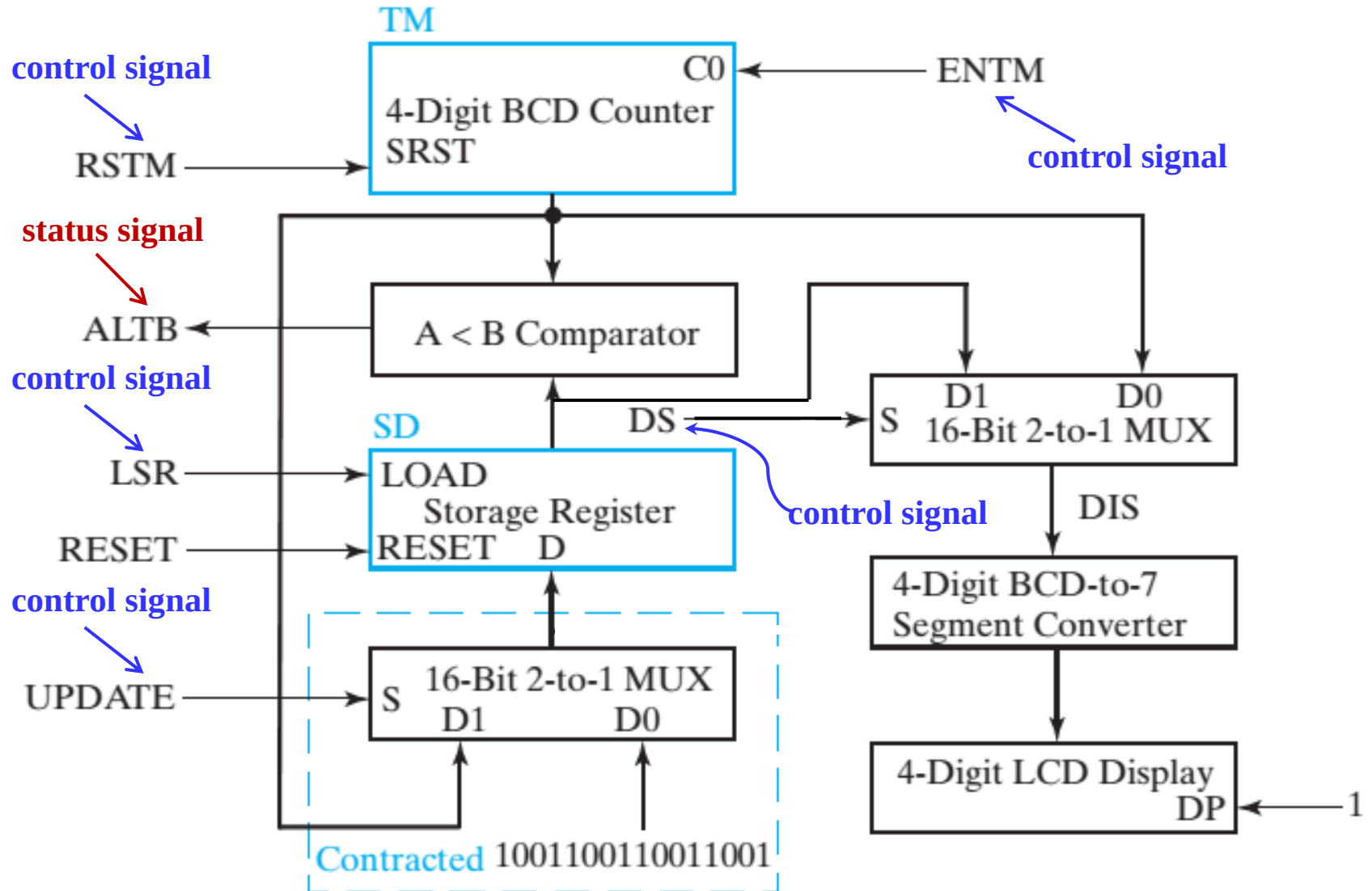
Action or Status	Control or Status Signals	Meaning for Values 1 and 0
$TM \leftarrow (0000)_{BCD}$ RSTM: $TM \leftarrow (0000)_{BCD}$	RSTM	1: Reset TM to 0 (synchronous reset) 0: No reset of TM
$TM \leftarrow (TM + 1)_{BCD}$ ENTM: $TM \leftarrow (TM + 1)$	ENTM	1: BCD count up TM by 1, 0: hold TM value
$SD \leftarrow (9999)_{BCD}$ LSR·UPDATE: $SD \leftarrow (9999)_{BCD}$ $SD \leftarrow TM$ LSR·UPDATE: $SD \leftarrow TM$	UPDATE LSR	0: Select 1001100110011001 for loading SD 1: Select TM for loading SD 1: Enable load SD, 0: disable load SD
$DIS = TM$ $\overline{DS}$: $DIS \leftarrow TM$ $DIS = SD$ DS: $DIS \leftarrow SD$	DS	0: Select TM for DIS 1: Select SD for DIS
$TM < SD$ $TM \geq SD$	ALTB	1: TM less than SD 0: TM greater than or equal to SD

status signal

Determination of Internal Control/Status Signals

- **TM – Timer**
 - Reset to 0000: RSTM
 - Enable to Count Up: ENTM
- **SD – Shortest Dash**
 - Load SD: LSR = 1;
 - Select input 9999: UPDATE = 0
 - Select input TM: UPDATE = 1
- **DIS – Display ($B_1, B_0, DP, B_{-1}, B_{-2}$)**
 - Select input TM: DS = 0
 - Select input SD: DS = 1
- **Compare TM and SD (**Status**)**
 - TM < SD: ALTB = 1
 - TM $\geq$ SD: ALTB = 0

DASHWATCH Datapath (Multiplexer Approach)



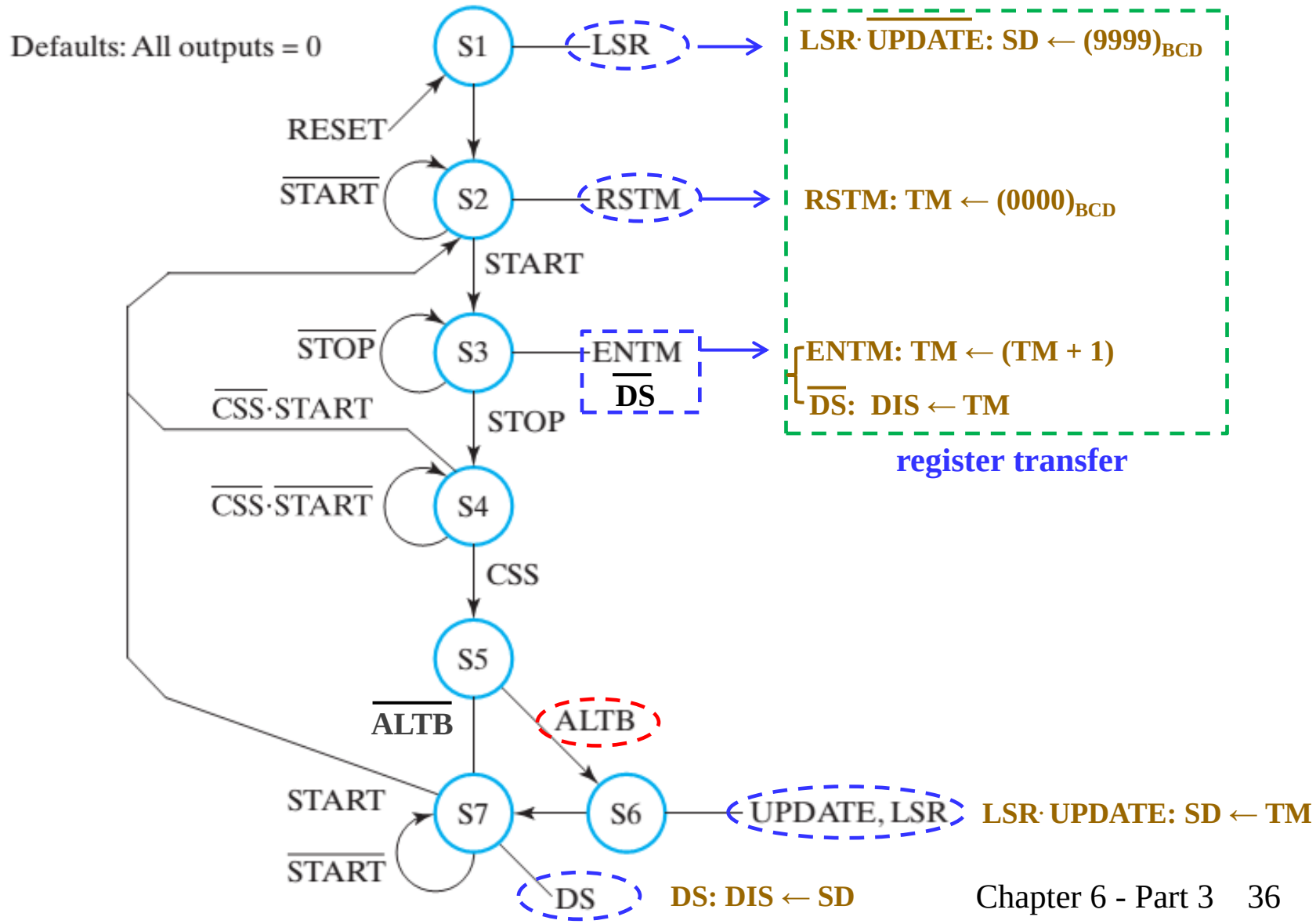
DASHWATCH – Datapath Development

- **TM: 4-digit BCD Counter with Synchronous Reset**
 - Based on previous BCD adder digit design
 - synchronous reset SRST added
 - SRST = RSTM
 - C0 (Incoming carry) = ENTM
- **A < B Comparator**
 - Compares TM to SD
 - Designed as left-to-right iterative cell array with output C0
- **SD: Standard 16-bit parallel load register**
 - LOAD = LSR
 - Contracted standard 2-way, 16-bit multiplexer used to select between 9999_{BCD} and TM as parallel load input D
 - S = UPDATE

DASHWATCH – Datapath Development – Display Logic

- **2-way 16-bit multiplexer**
 - Selects between TM and SD
 - $S = DS$
- **4-digit BCD-to-7 Segment Converter**
 - Uses previous design
- **4-digit 7-Segment Display with Decimal Point**
 - 2-digit fractional part
 - Decimal Point control = DP
 - $DP = 1$

DASHWATCH – State Machine Diagram with Control Signal Outputs Replacing Register Transfers

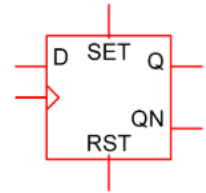


DASHWATCH – FF Input Equations

- One-Hot State Assignment with D flip-flops**

– 7 bits: $Q_7Q_6Q_5Q_4Q_3Q_2Q_1$

- State S1 entered only by using asynchronous SET**



$$D_{S1} = S1(t + 1) = 0 \quad \text{state bits for } D_{s2}: S1-Q_1, S2-Q_2, S4-Q_4, S7-Q_7$$



$$D_{S2} = S2(t + 1) = S1 + S2 \cdot \overline{START} + S4 \cdot \overline{CSS} \cdot START + S7 \cdot START$$

$$D_{S3} = S3(t + 1) = S2 \cdot START + S3 \cdot \overline{STOP}$$

$$D_{S4} = S4(t + 1) = S3 \cdot STOP + S4 \cdot \overline{CSS} \cdot \overline{START}$$

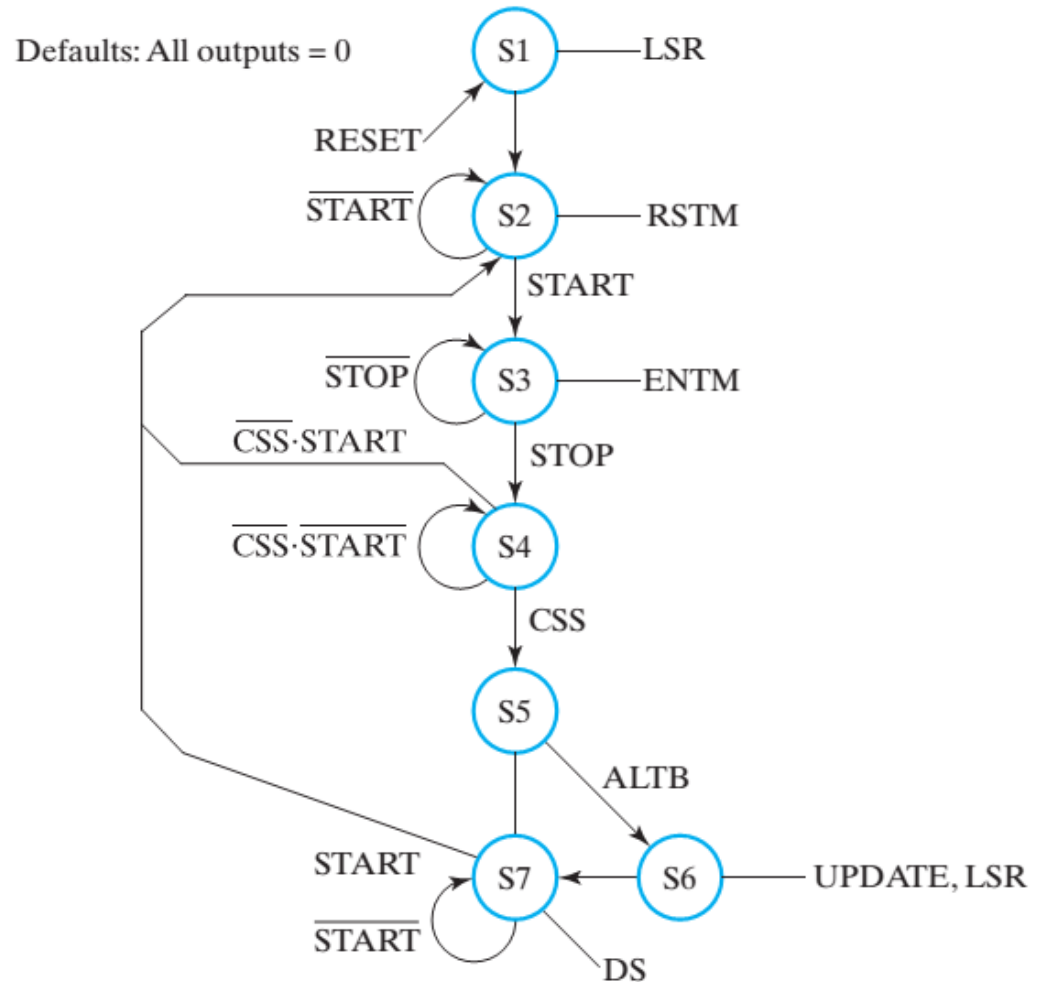
$$D_{S5} = S5(t + 1) = S4 \cdot CSS$$

$$D_{S6} = S5 \cdot ALTB$$

$$D_{S7} = S7(t + 1) = S5 \cdot \overline{ALTB} + S6 + S7 \cdot \overline{START}$$

DASHWATCH – Output Equations

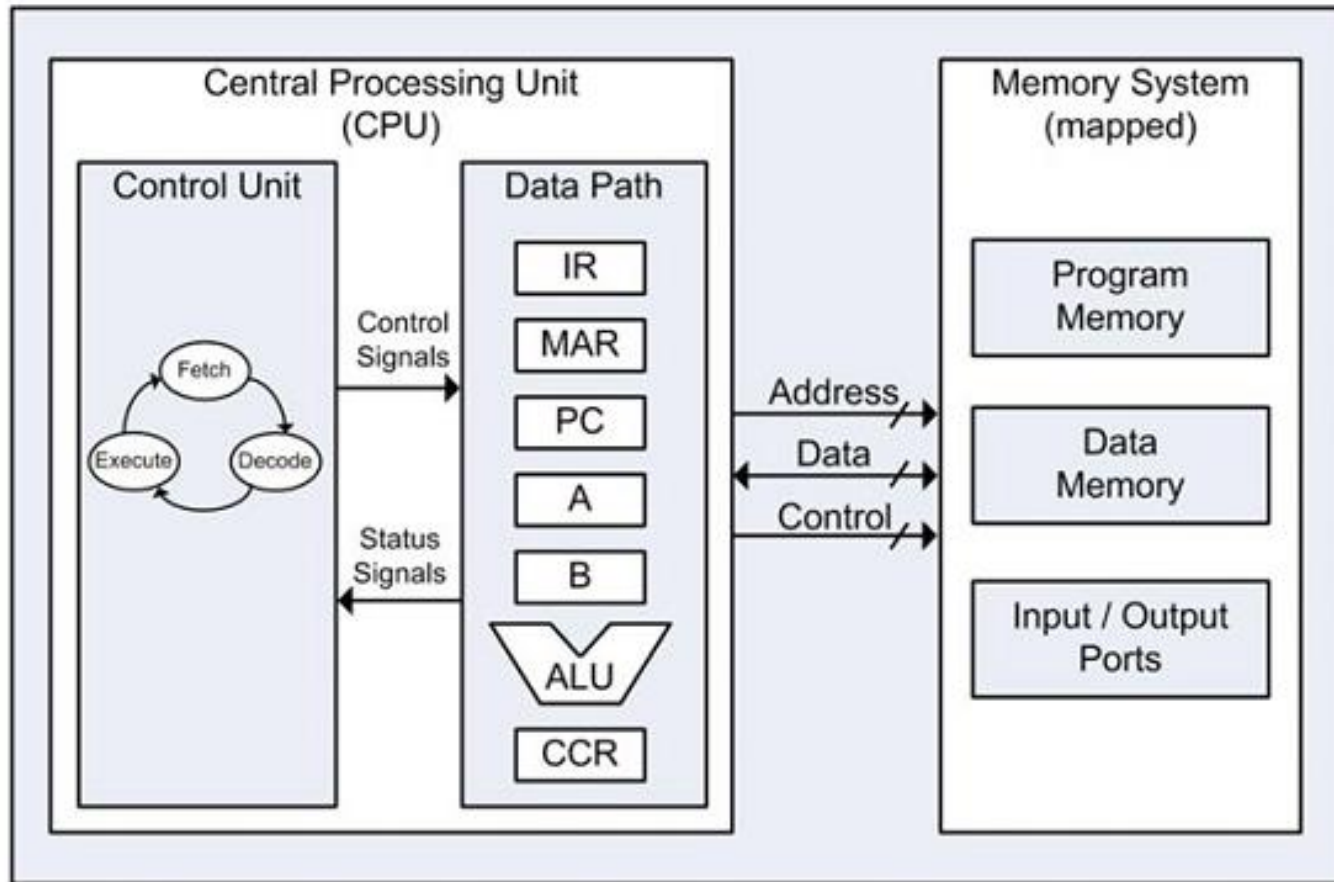
$$\begin{aligned} LSR &= S1 + S6 \\ RSTM &= S2 \\ ENTM &= S3 \\ UPDATE &= S6 \\ DS &= S7 \end{aligned}$$



Programmable and Non-Programmable Systems

- **Non-programmable System: Specific system**
 - the control unit does not deal with fetching and executing instructions.
 - There is no PC or similar register in such a system.
 - Instead, the control unit determines the operations to be performed and the sequence of those operations, based on its inputs and the status bits from the datapath.
- **Programmable System: General system**
 - A portion of the input consists of a sequence of **instructions** called a **program**.
 - Typically stored in a memory and addressed by a **program counter**.
 - The **Control Unit** is responsible for fetching and executing these instructions.

Microprogrammed Control

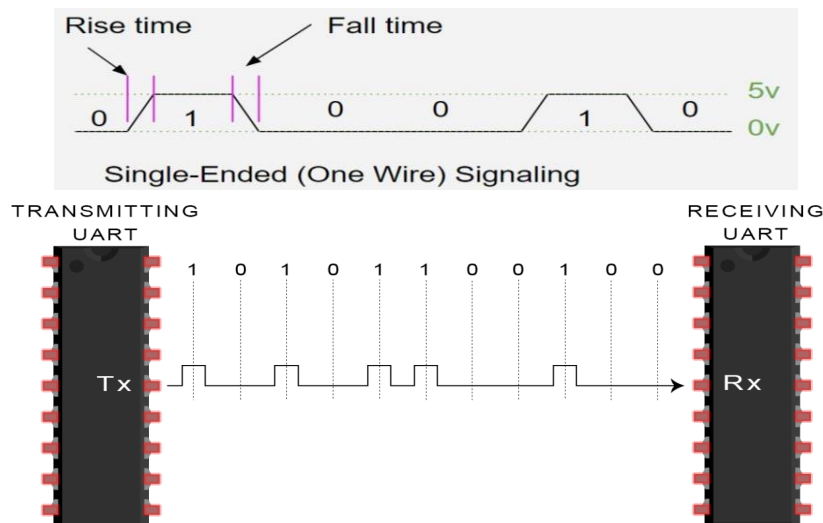


Overview

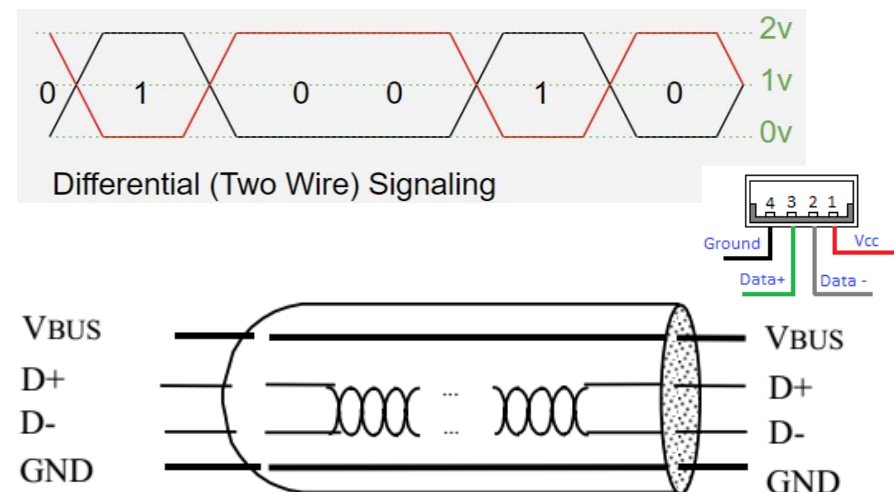
- **Part 1 – Registers, Microoperations and Implementations**
- **Part 2 – Counters, register cells, buses, & serial operations**
 - **Microoperations on single register (continued)**
 - **Counters**
 - **Serial transfers and microoperations**
 - **Register cell design**
- **Part 3 – Control of Register Transfers**

Serial Transfers and Microoperations

- A digital system is said to operate in a **serial mode** when information in the system is **transferred or manipulated one bit at a time**.
- **Serial transfer method** is in contrast to **parallel transfer**, in which all the bits of the register are transferred at the same time.



Serial data transfer by UART

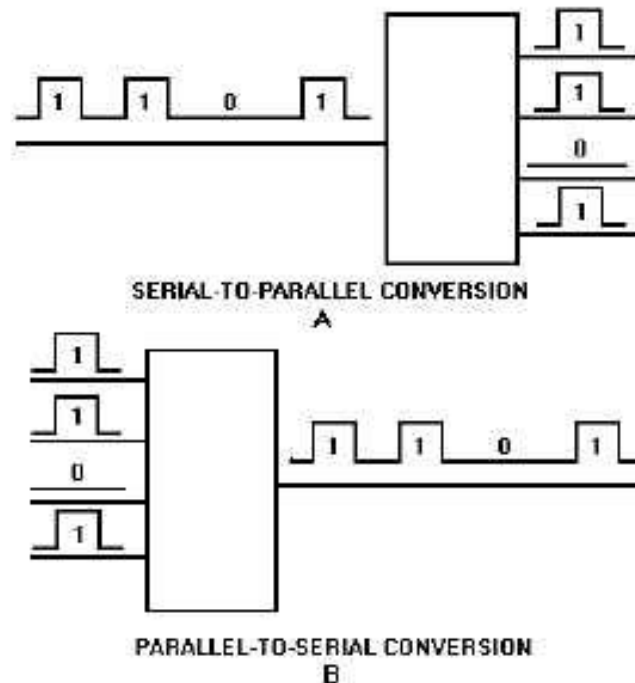


Serial data transfer by USB 2.0

Serial Transfers and Microoperations

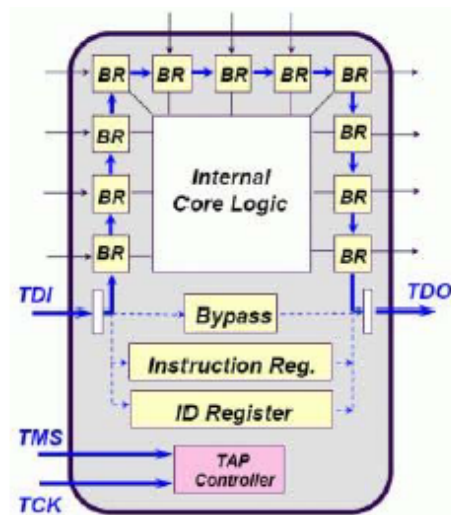
(continued)

- **Serial Transfers**
 - Used for “narrow” transfer paths
- **Example 1: Telephone or cable line**
 - **Serial-to-Parallel** conversion at destination
 - **Parallel-to-Serial** conversion at source

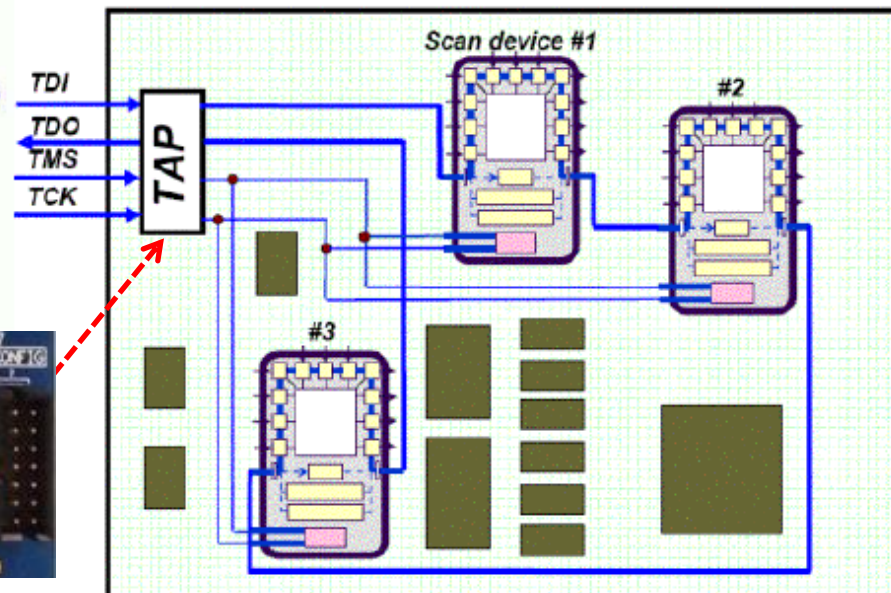


Serial Transfers and Microoperations (continued)

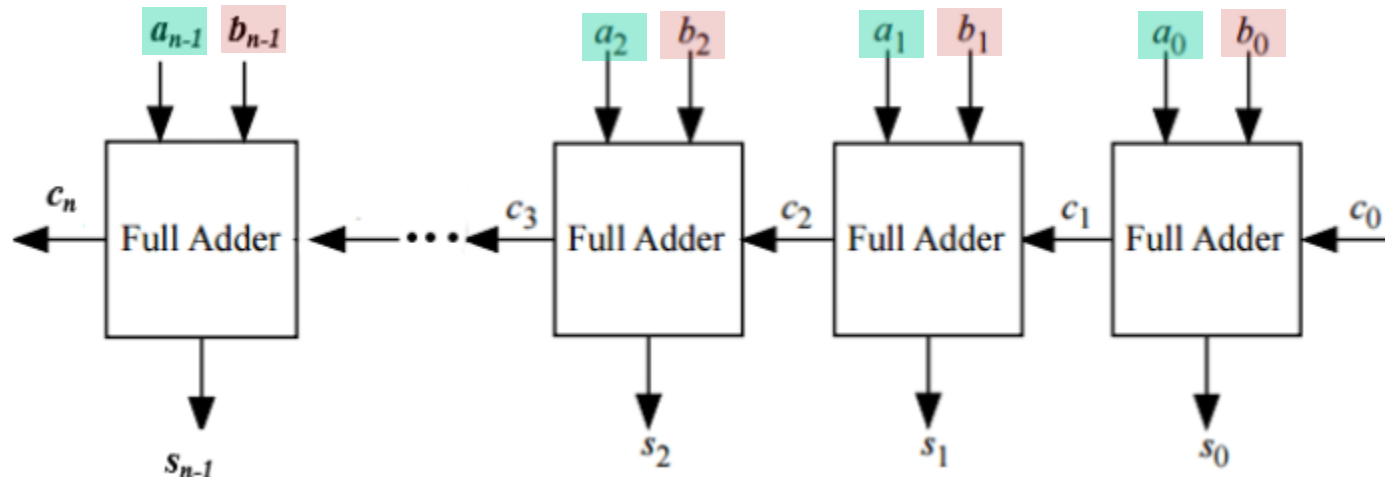
- **Example 2: Initialization and capture of the contents of many flip-flops for test purposes**
 - Add shift function to all flip-flops and form large shift register
 - Use shifting for simultaneous initialization and capture operations



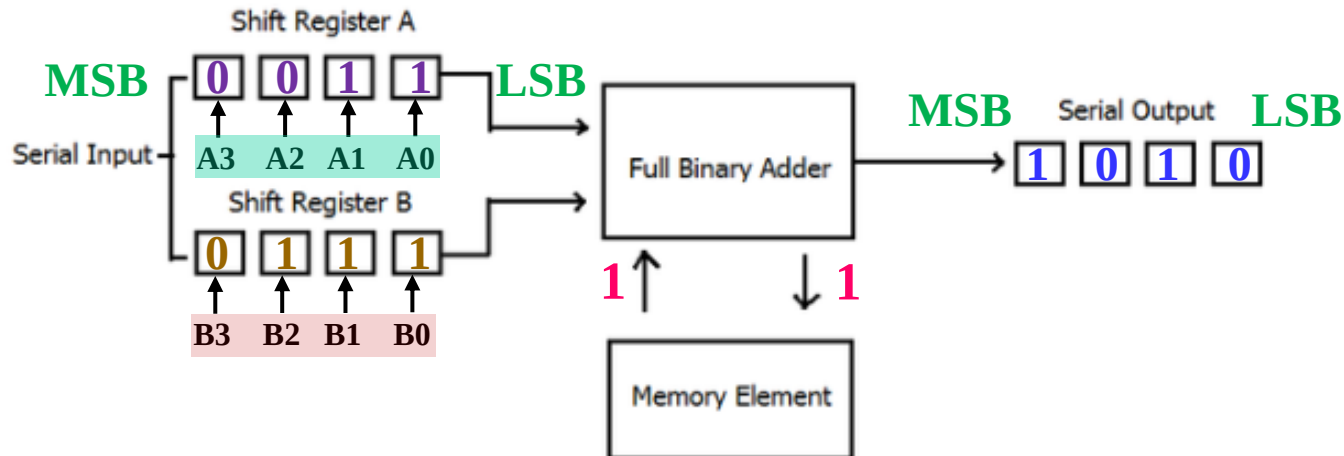
USB-JTAG



Parallel Adder vs. Serial Adder

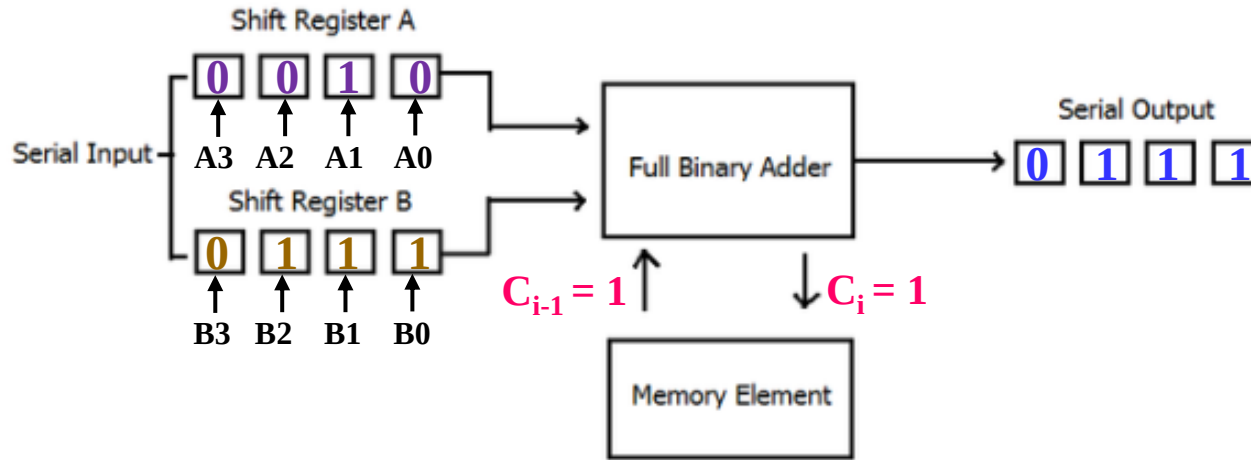


Parallel Adder



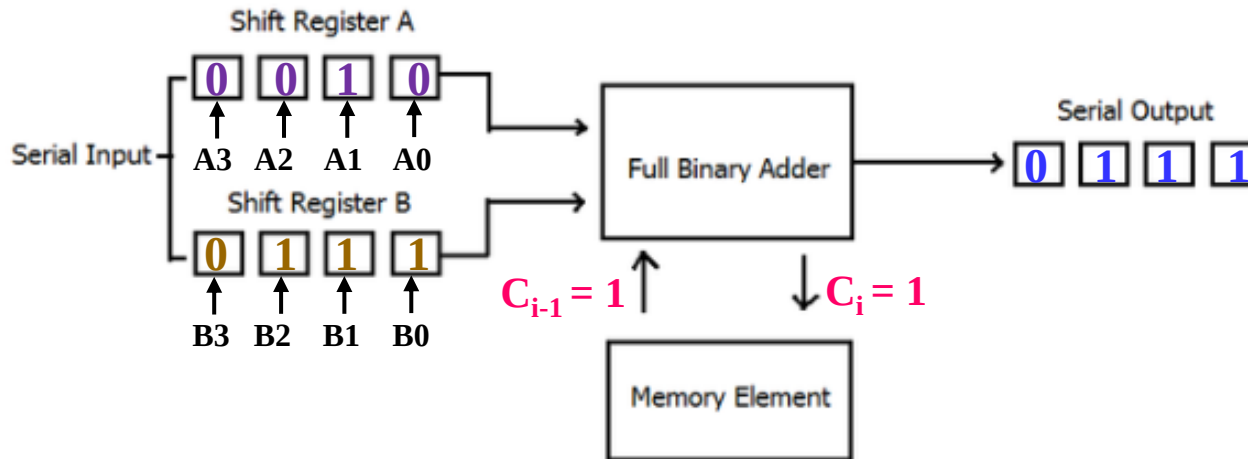
Serial Adder

Serial Microoperation Example: Serial Adder



- Serial binary adder is a **sequential circuit** that performs the addition of two binary numbers in serial form.
- Two states are abstracted to remember the carry:
 - G: carry in = 0
 - H: carry in = 1

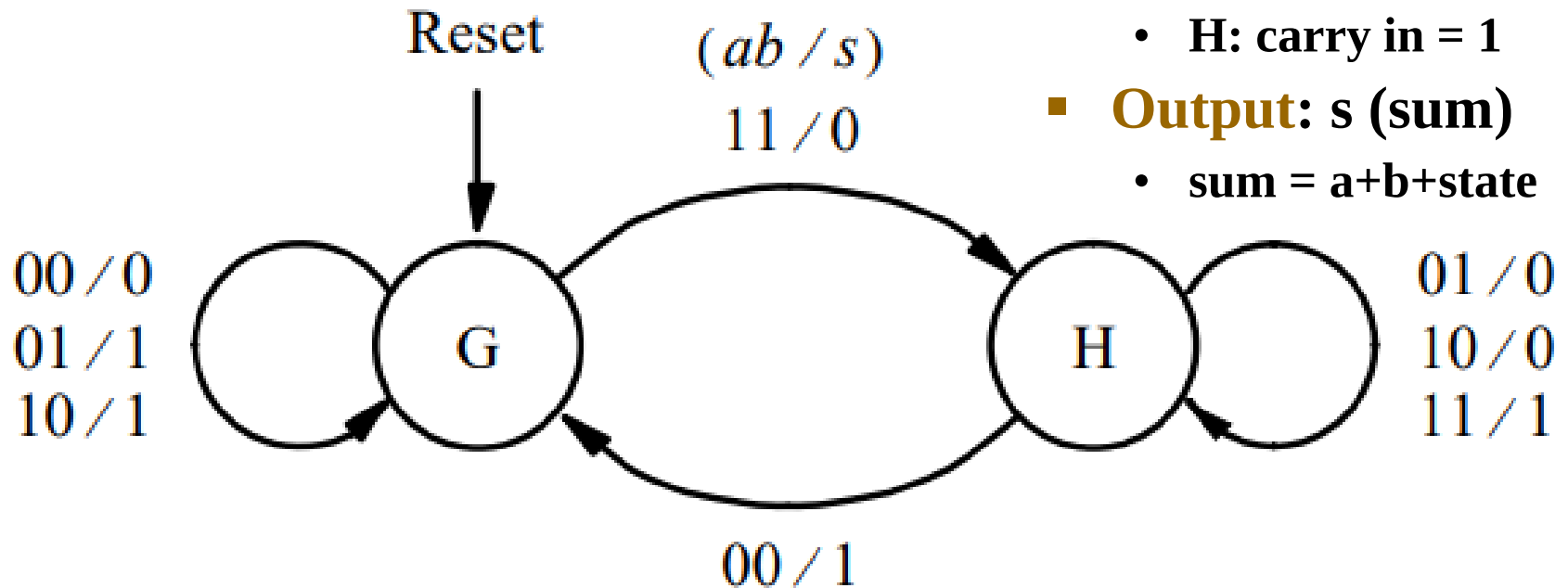
Serial Adder Specification—Mealy Model



- Serial adder specification:
 - **Input:** a, b
 - **State:**
 - G: carry in = 0
 - H: carry in = 1
 - **Output:** s (sum)
 - $\text{sum} = a + b + \text{state}$ (Mealy Model)

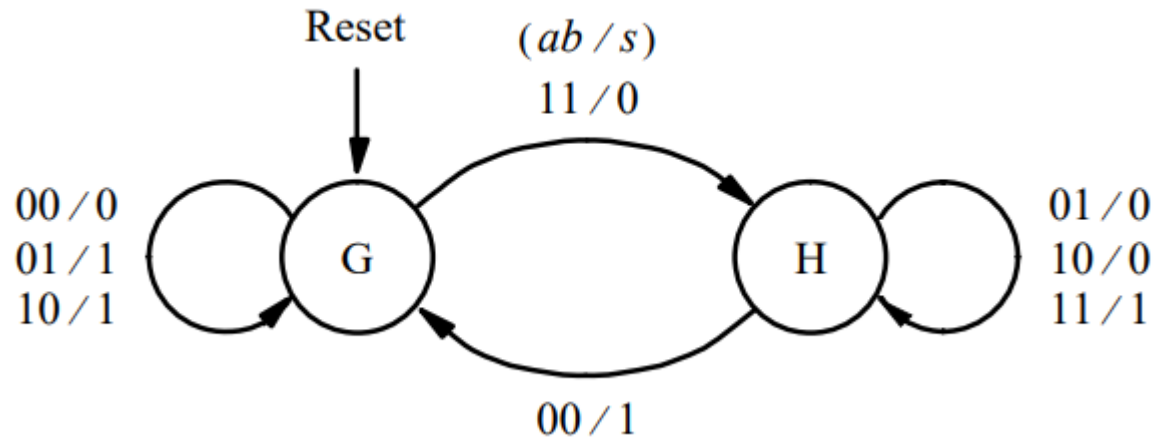
Mealy Model for Serial Adder

- **Input:** a, b
- **State:**
 - G: carry in = 0
 - H: carry in = 1
- **Output:** s (sum)
 - $\text{sum} = a + b + \text{state}$



State diagram of Mealy type serial adder

State Table for Serial Adder



State table



Present state	Next state				Output s			
	$ab = 00$	01	10	11	00	01	10	11
G	G	G	G	H	0	1	1	0
H	G	H	H	H	1	0	0	1

State Assignment for Serial Adder

Present state	Next state				Output s			
	$ab=00$	01	10	11	00	01	10	11
G	G	G	G	H	0	1	1	0
H	G	H	H	H	1	0	0	1

State assignment: G = 0, H = 1



Present state	Next state				Output			
	$ab=00$	01	10	11	00	01	10	11
y	Y				s			
0	0	0	0	1	0	1	1	0
1	0	1	1	1	1	0	0	1

Input and Output Equation for Serial Adder

Present state y	Next state				Output			
	$ab=00$	01	10	11	00	01	10	11
	Y				s			
0	0	0	0	1	0	1	1	0
1	0	1	1	1	1	0	0	1

- Assume D flip-flop
- Input equation:
 - $D = ab + ay + by$
- Output equation:
 - $S = a \oplus b \oplus y$

		a, b			
		00	01	11	10
y	0	0	0	1	0
	1	0	1	1	1

		a, b			
		00	01	11	10
y	0	0	1	0	1
	1	1	0	1	0

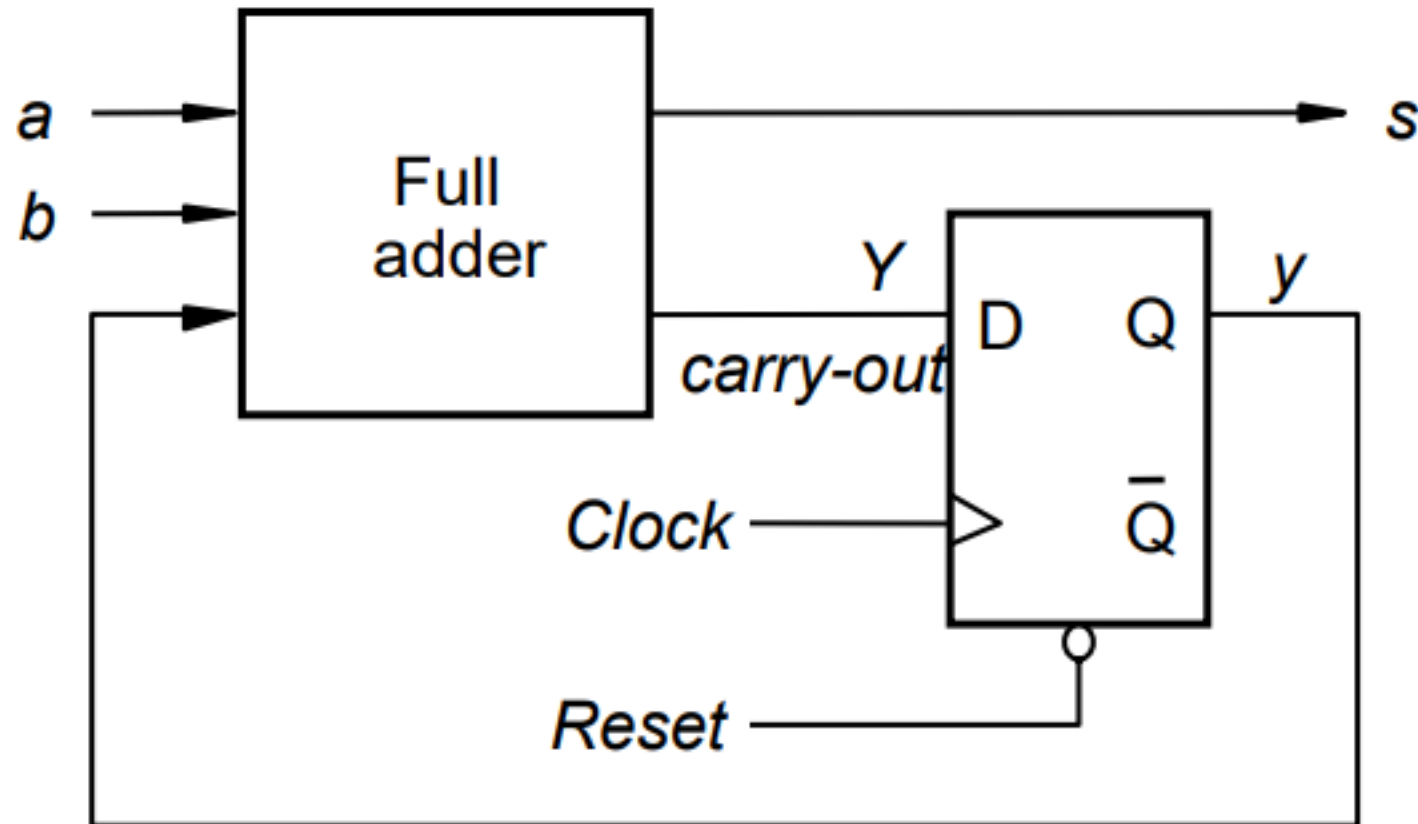
Circuit for the Mealy Model Serial Adder

- Input equation:

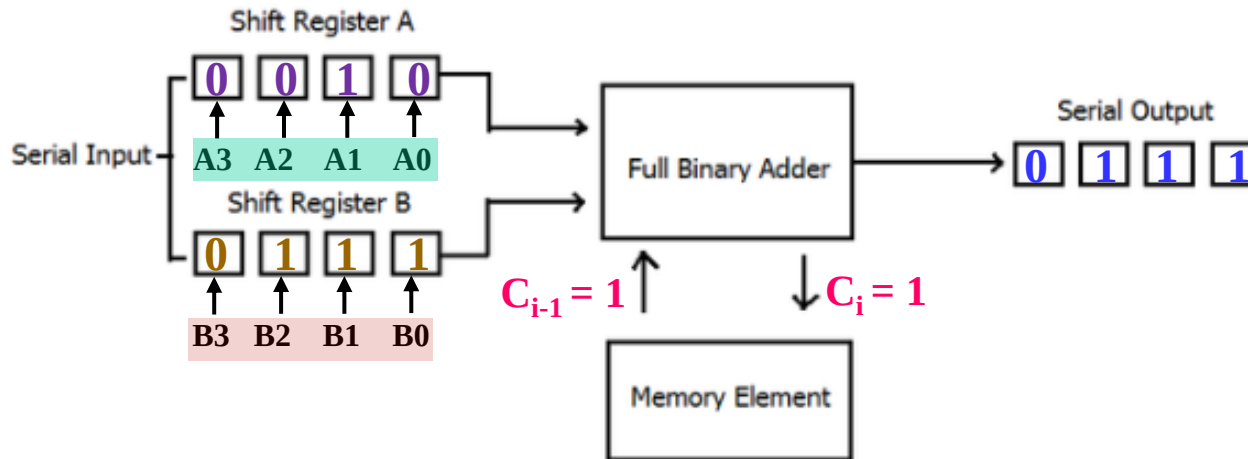
- $D = ab + ay + by$

- Output equation:

- $S = a \oplus b \oplus y$



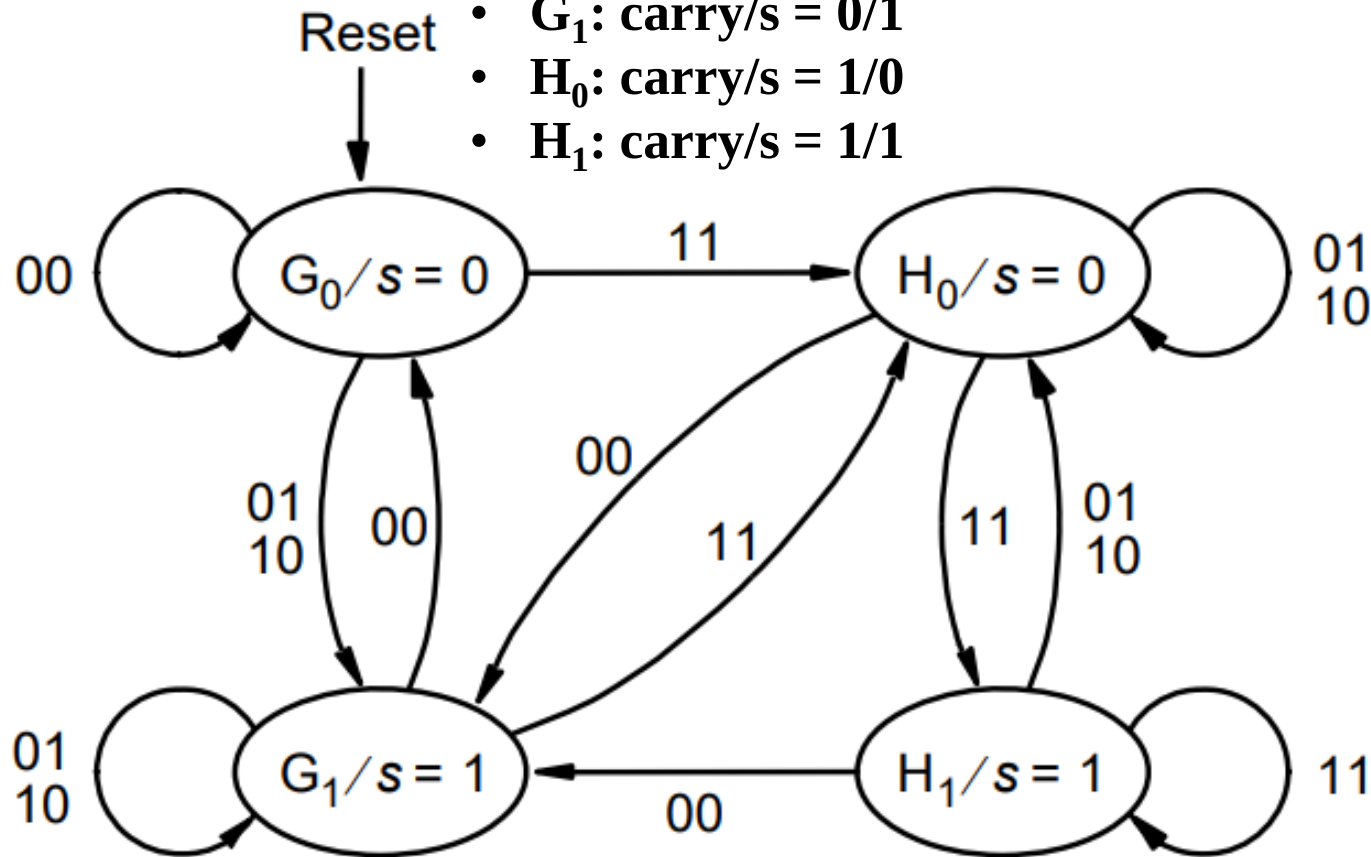
Serial Adder Specification—Moore Model



- **Serial adder specification:**
 - **Input:** a, b
 - **State:**
 - G_0 : carry = 0, sum = 0
 - H_0 : carry = 1, sum = 0
 - G_1 : carry = 0, sum = 1
 - H_1 : carry = 1, sum = 1
 - **Output:** s (sum)
 - sum = state (Moore Model)

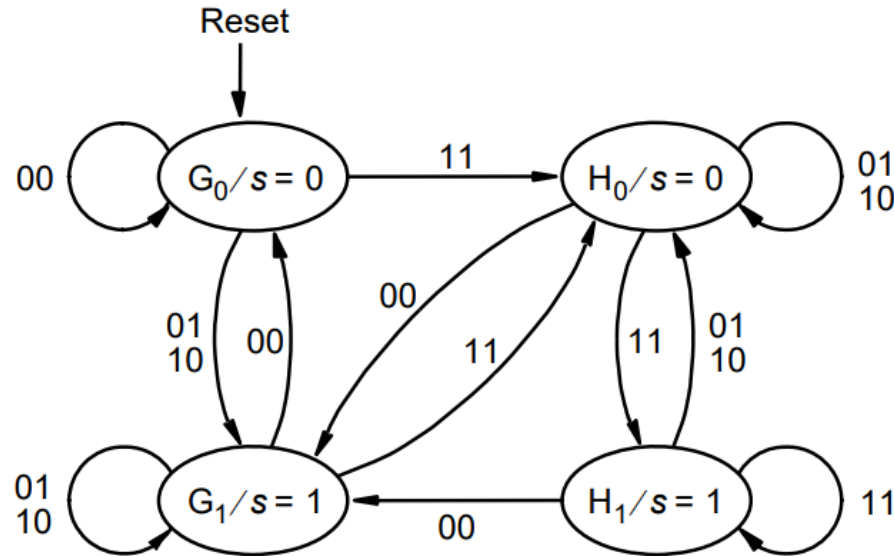
Moore Model for Serial Adder

- **Input:** a, b
- **State:**
 - G_0 : carry/s = 0/0
 - G_1 : carry/s = 0/1
 - H_0 : carry/s = 1/0
 - H_1 : carry/s = 1/1



State diagram of Moore type serial adder

State Table for Serial Adder



State table



Present state	Nextstate				Output s
	$ab = 00$	01	10	11	
G_0	G_0	G_1	G_1	H_0	0
G_1	G_0	G_1	G_1	H_0	1
H_0	G_1	H_0	H_0	H_1	0
H_1	G_1	H_0	H_0	H_1	1

State Assignment for Serial Adder

Present state	Next state				Output <i>s</i>
	<i>ab</i> = 00	01	10	11	
G_0	G_0	G_1	G_1	H_0	0
G_1	G_0	G_1	G_1	H_0	1
H_0	G_1	H_0	H_0	H_1	0
H_1	G_1	H_0	H_0	H_1	1



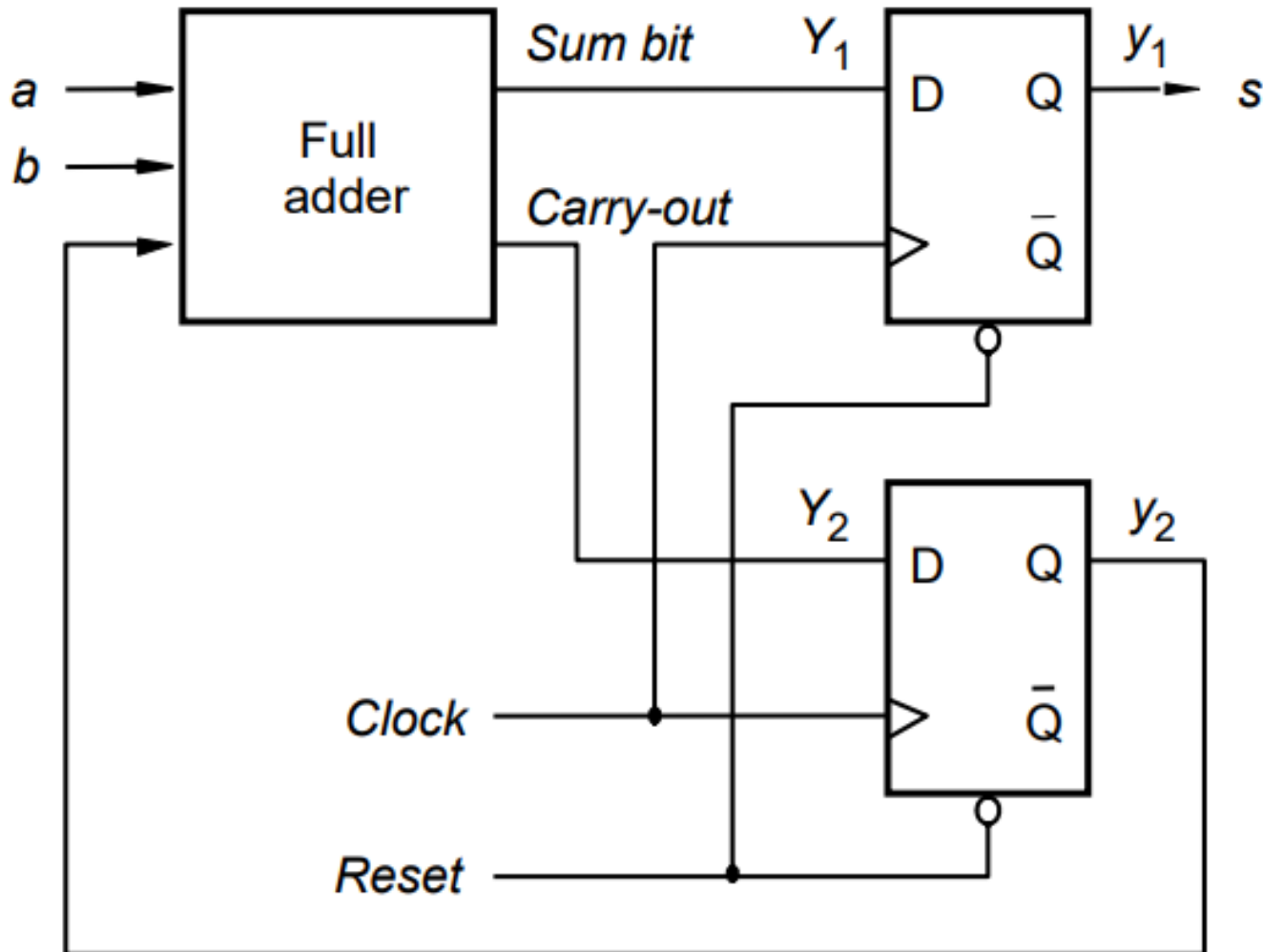
Present state y_2y_1	Next state				Output s
	$ab = 00$	01	10	11	
	Y_2Y_1				
0 0	0 0	0 1	0 1	1 0	0
0 1	0 0	0 1	0 1	1 0	1
1 0	0 1	1 0	1 0	1 1	0
1 1	0 1	1 0	1 0	1 1	1

State assignment:

- $G_0 = 00$
- $G_1 = 01$
- $H_0 = 10$
- $H_1 = 11$

- Assume D flip-flops
- Input equation:
 - $D_2 = ab + ay_2 + by_2$
 - $D_1 = a \oplus b \oplus y_2$
- Output equation:
 - $S = y_1$

Circuit for the Moore Model Serial Adder



Assignments

Reading:

- 6-7, 6-9, 6-10

Problem assignment:

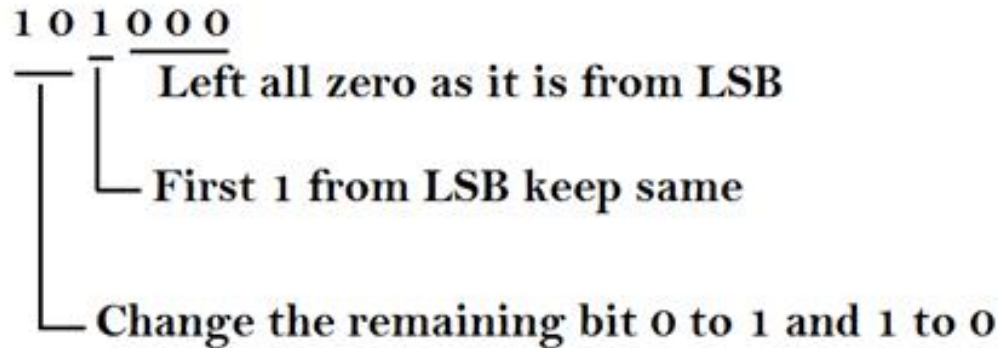
- 6-23

Practice

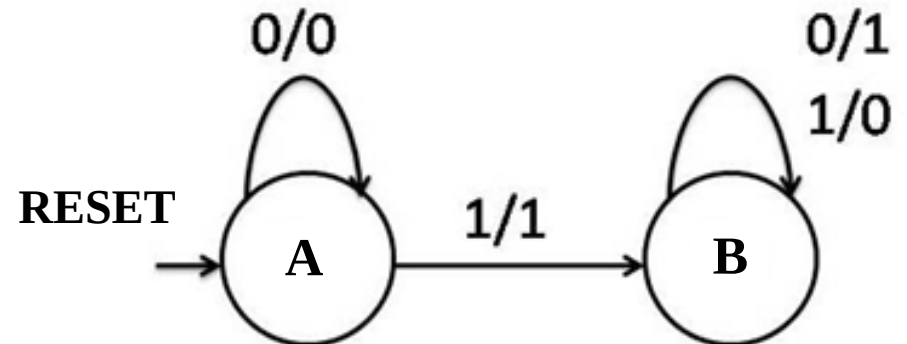
Question: Design a one-input, one-output serial 2's complementer using D flip-flops. The circuit accepts a string of bits starting from the least significant bit (LSB) and produces the 2's complement at the output.

- e.g., 001010 $\rightarrow$ 110110
- The circuit includes an asynchronous reset to initialize and terminate the operation.

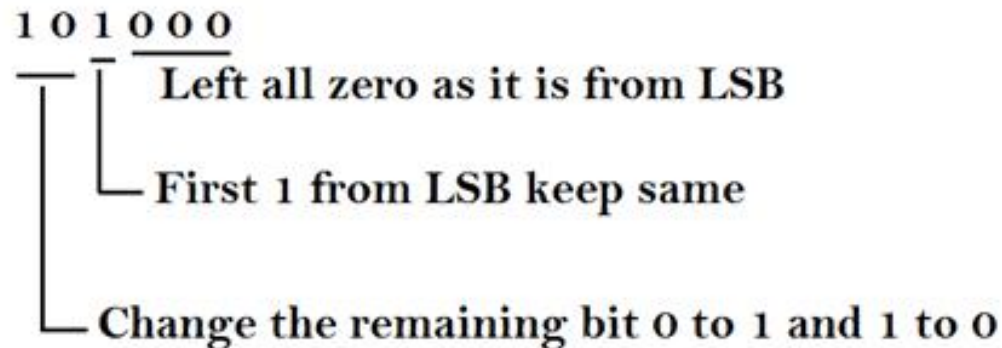
Practice (continued)



- **Input: x**
- **State:**
 - A: wait for a “1”
 - B: got “1”
- **Output: Y (Mealy Model)**
 - 0, if state = A and x = 0
 - 1, if state = A and x = 1
 - 1, if state = B and x = 0
 - 0, if state = B and x = 1



Practice (continued)



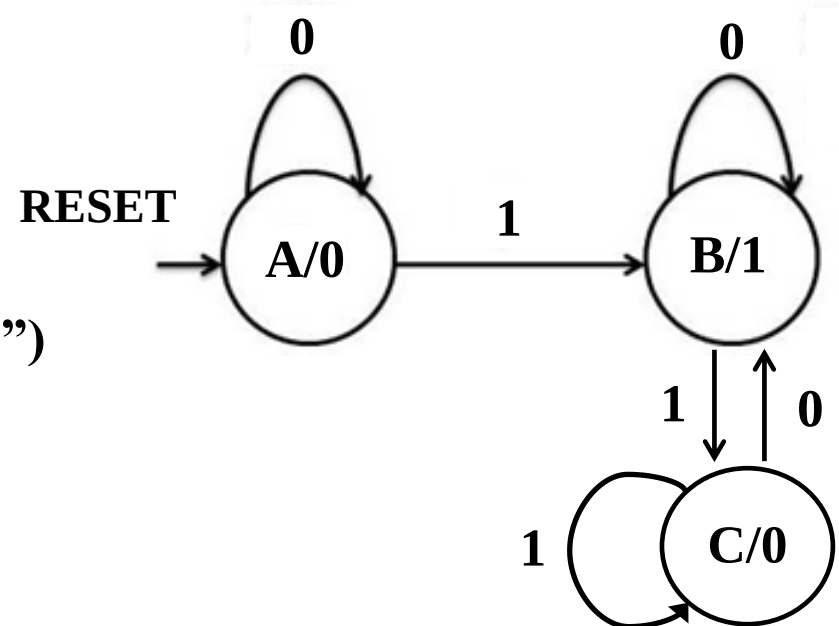
■ **Input:** x

■ **State:**

- A: wait for a “1”
- B: got first “1” or
(got first “1” and got a “0”)
- C: got first “1” and got “1”

■ **Output:** Y (**Moore Model**)

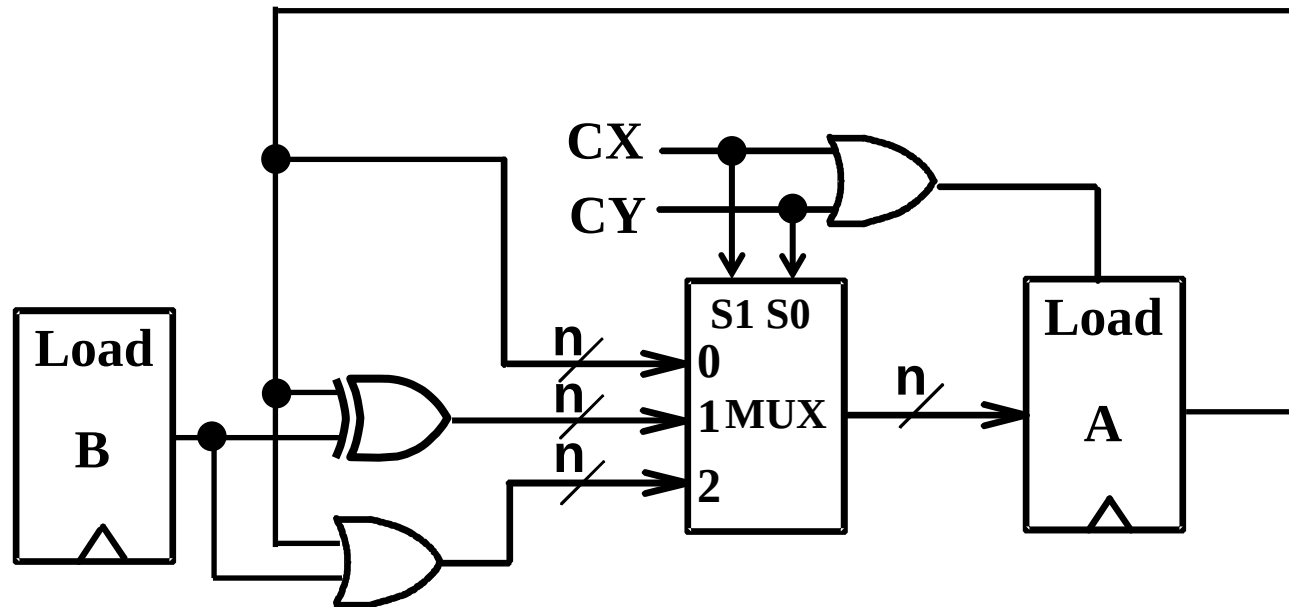
- output = state

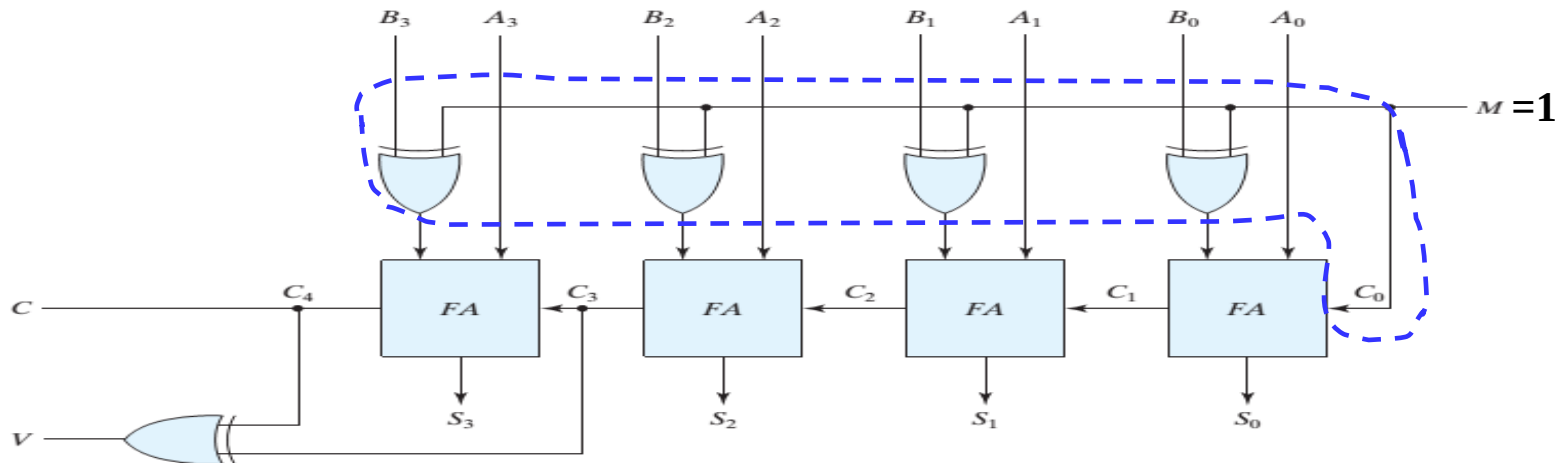


Example 1: Register Cell Design (continued)

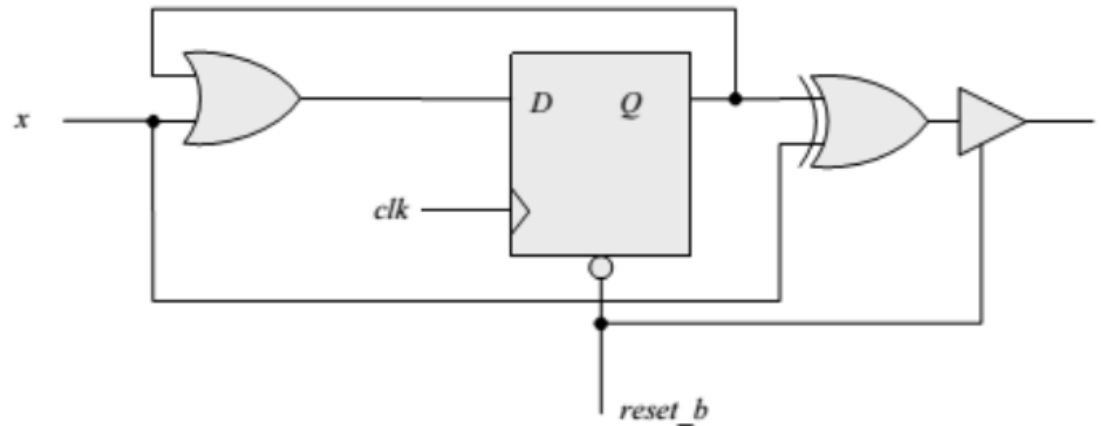
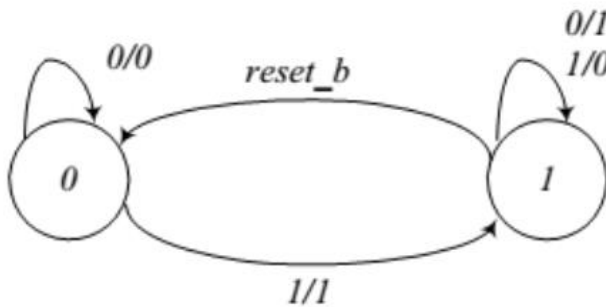
■ Register A (n-bits) Specification:

- **CX:** $A \leftarrow B \vee A$
- **CY:** $A \leftarrow B \oplus A$





parallel 2's complements

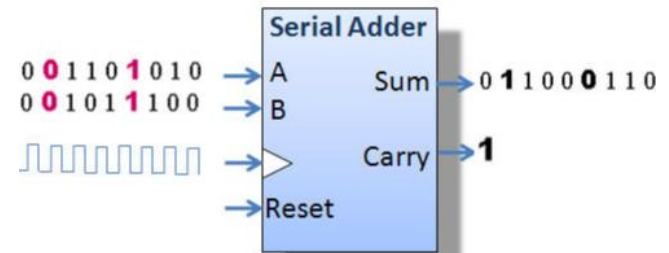


serial 2's complemeter

Serial Microoperations

- **Serial microoperations**

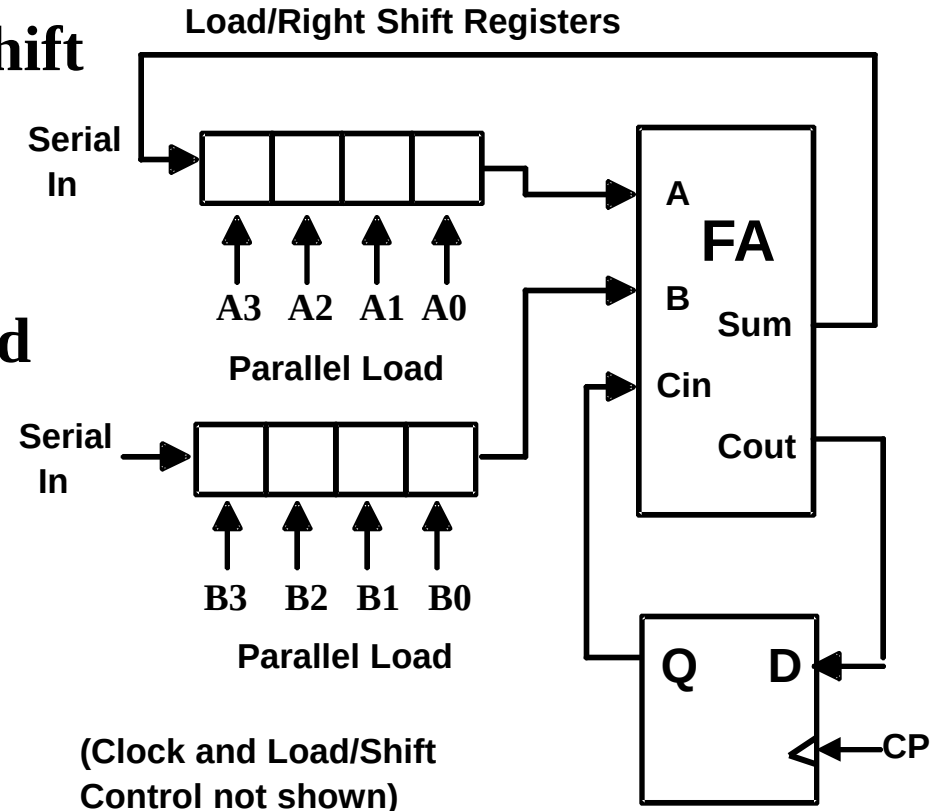
- **Example: Addition**



- By using two shift registers for operands, a full adder, and a flip flop (for the carry), we can add two numbers serially, starting at the least significant bit.
- Serial addition is a low cost way to add large numbers of operands, since a “tree” of full adder cells can be made to any depth, and each new level doubles the number of operands.
- Other operations can be performed serially as well, such as parity generation/checking or more complex error-check codes.

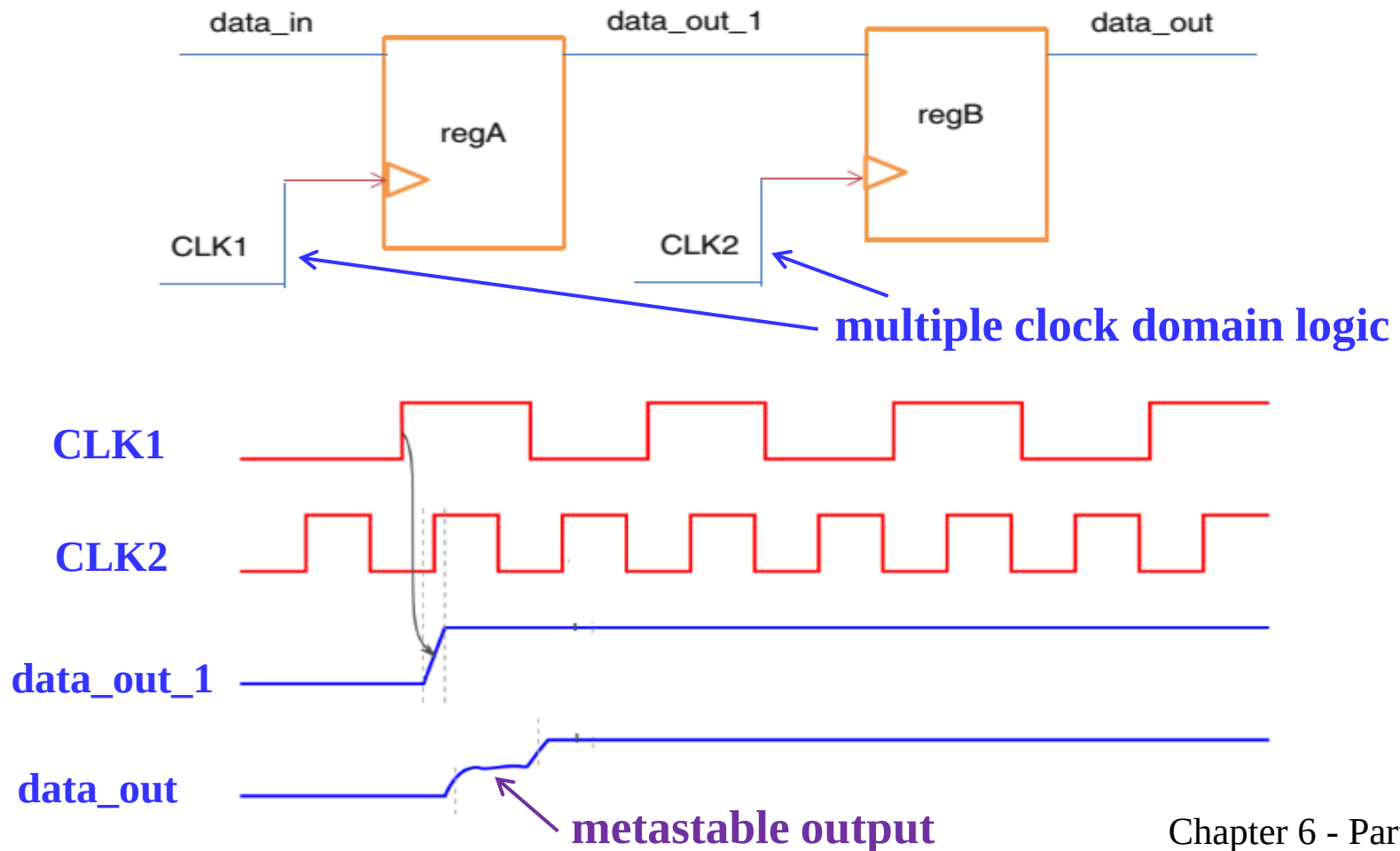
Serial Adder

- The circuit shown uses two shift registers for operands A(3:0) and B(3:0).
- A full adder, and one more flip flop (for the carry) is used to compute the sum.
- The result is stored in the A register and the final carry in the flip-flop
- With the operands and the result in shift registers, a tree of full adders can be used to add a large number of operands. Used as a common digital signal processing technique.



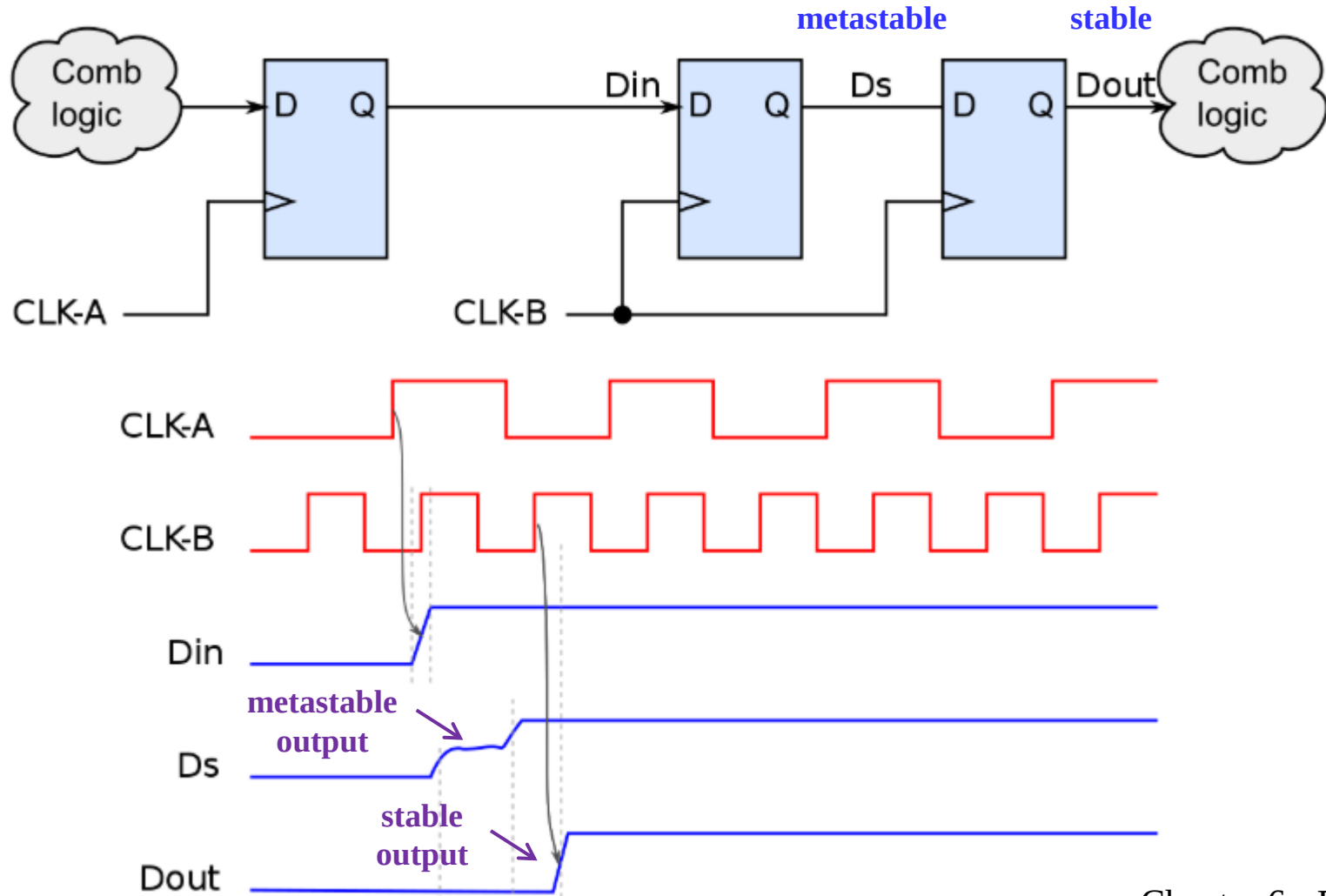
Appendix B: Signal Transfer across the Clock Boundary

- Passing data from one clock domain to another clock domain is difficult and error-prone task.



Signal Transfer across the Clock Boundary (continued)

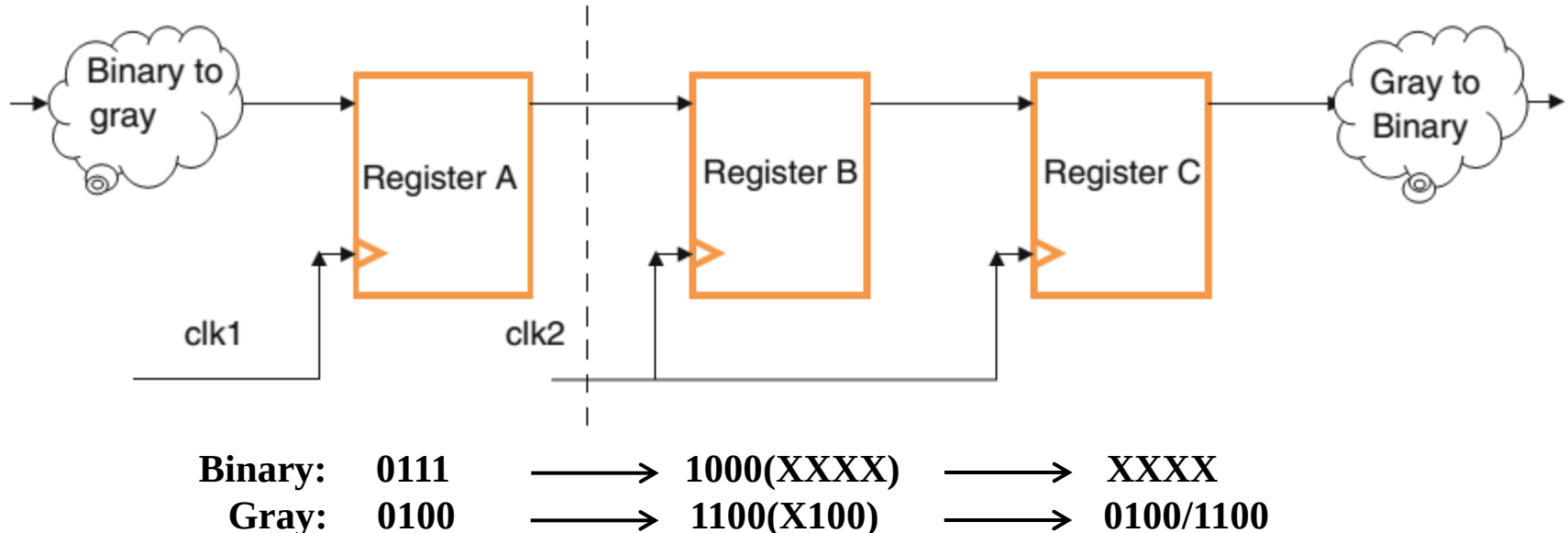
■ Two Flip-Flop Synchronizer



Signal Transfer across the Clock Boundary (continued)

■ Gray Encoding

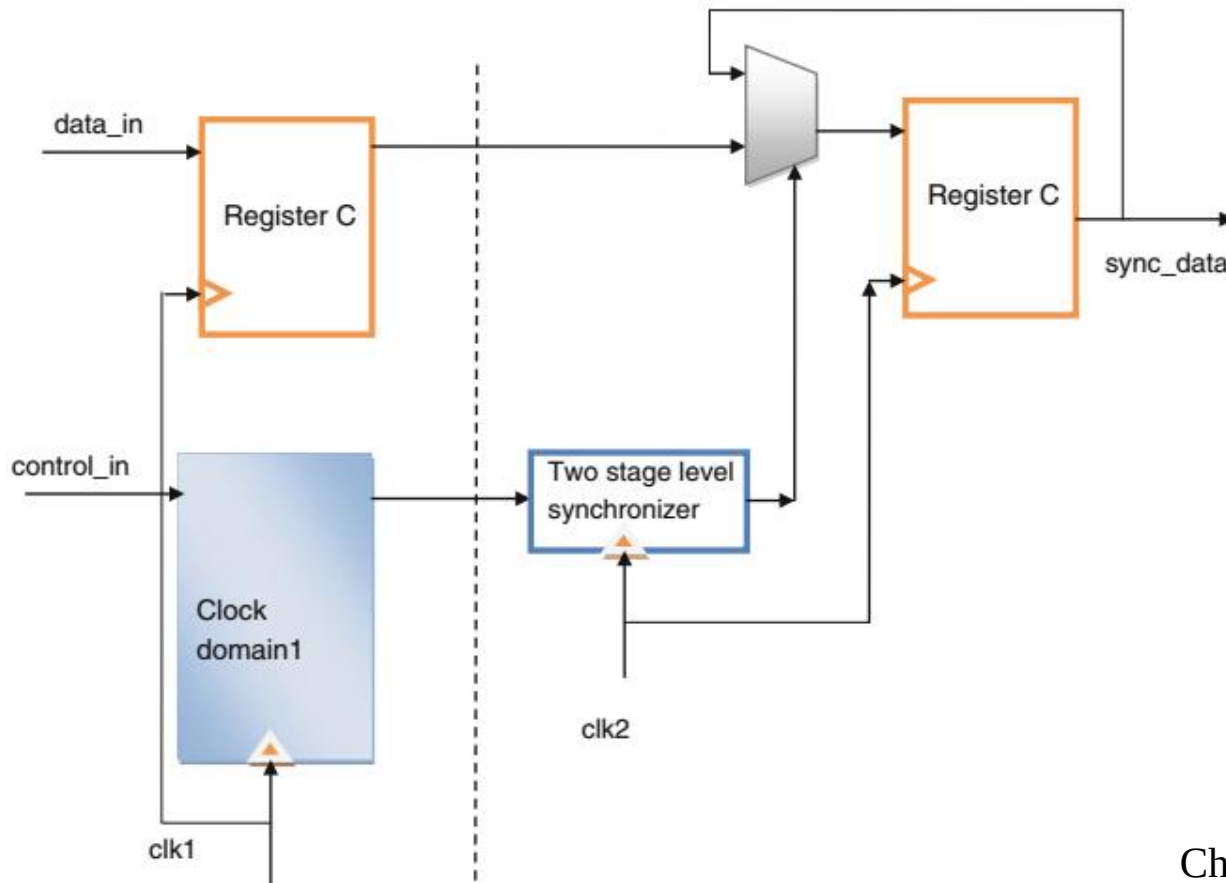
- While passing multiple bits of the data or control signals it is essential to use the gray encoding technique.
- Gray encoding is guaranteed to sample the one-bit change across the clocking boundary.



Signal Transfer across the Clock Boundary (continued)

■ MUX Synchronizer

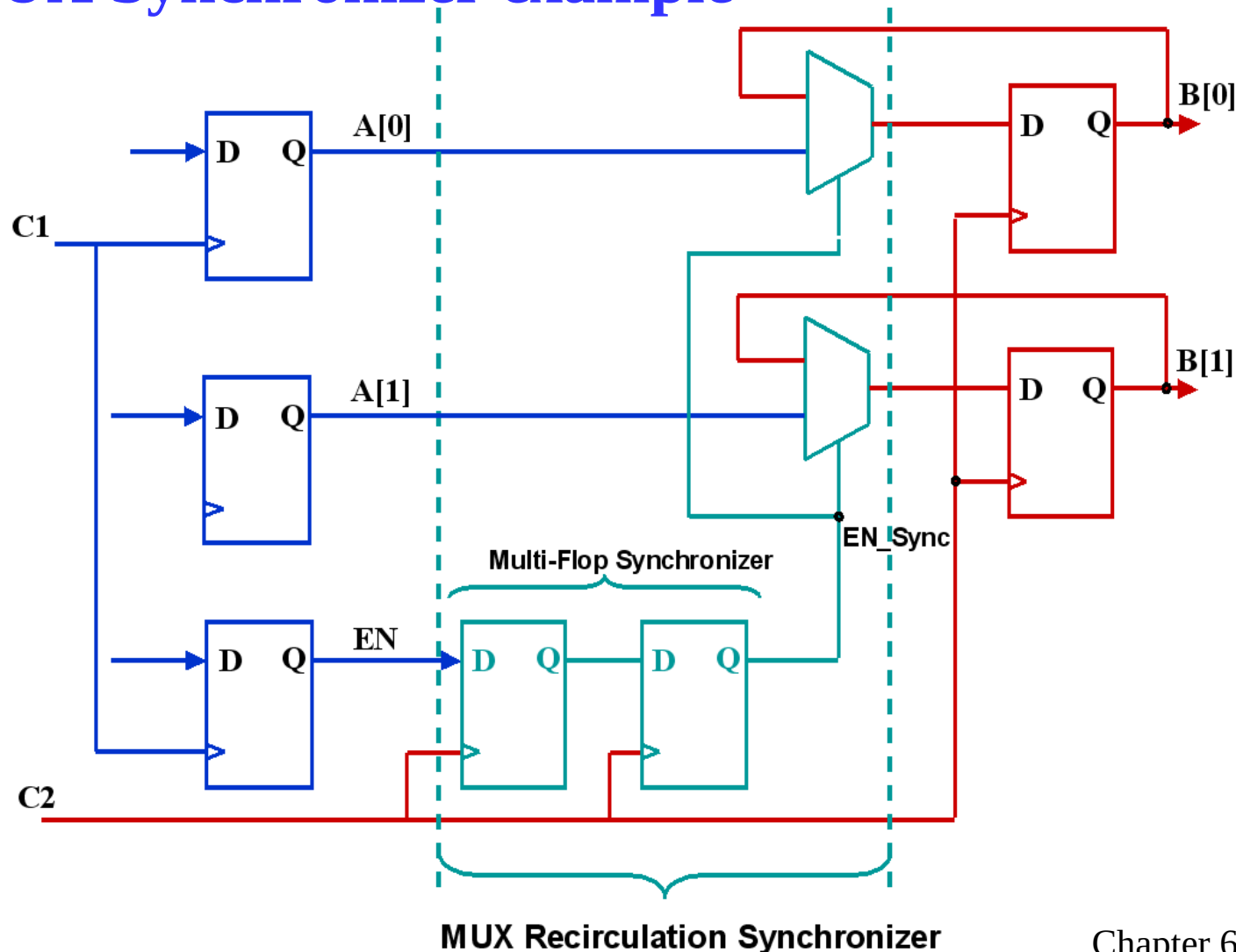
- Use the pair of the data and control signals while sending the information across clock domain.



Signal Transfer across the Clock Boundary

(continued)

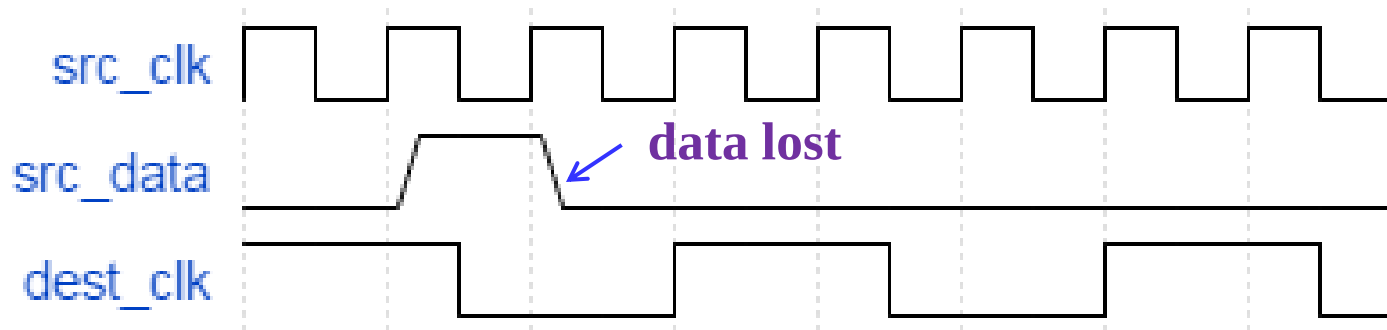
■ MUX Synchronizer example



Signal Transfer across the Clock Boundary (continued)

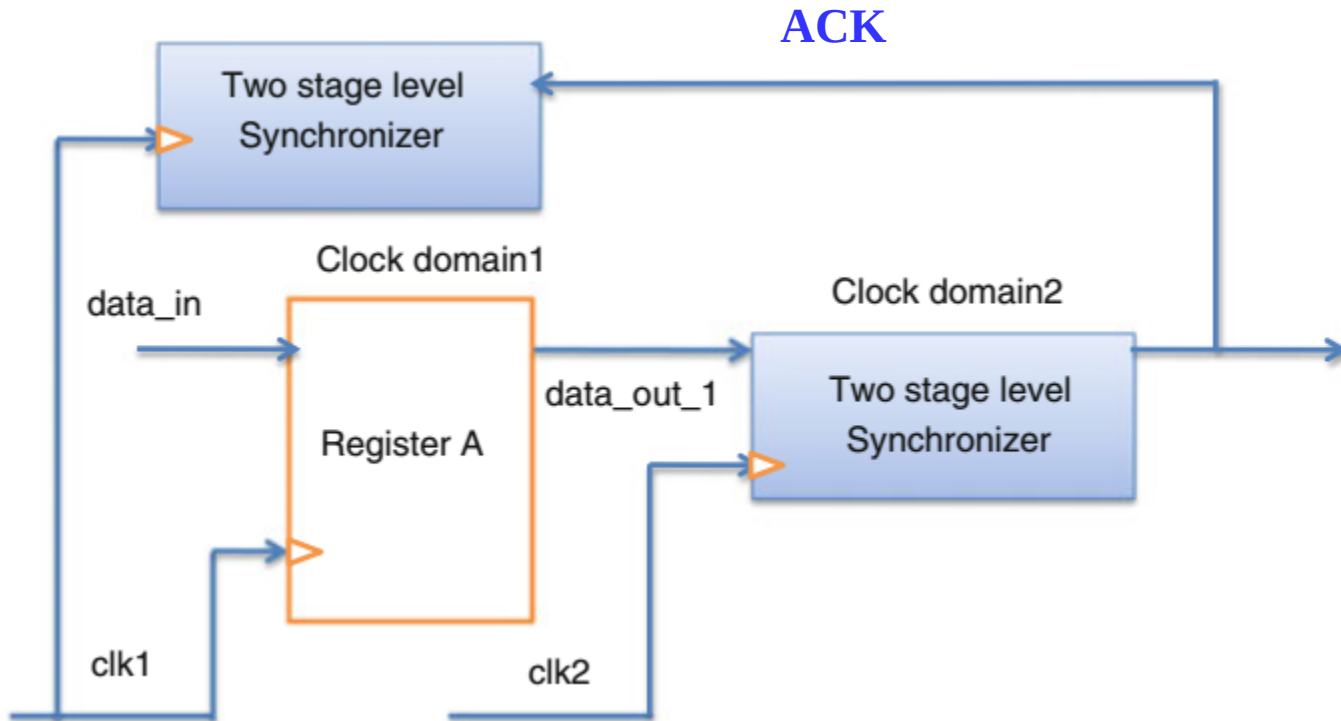
- **Two scenarios of clock domain crossing**
 - **Synchronizing slow signals into fast clock domain**

Using “**open-loop**” synchronizers without acknowledgement.
 - **Synchronizing fast signals into slow clock domain**
 - Using pulse stretcher (“**open-loop**” synchronizers)
 - Using “**closed-loop**” synchronizers with acknowledgement.



Signal Transfer across the Clock Boundary (continued)

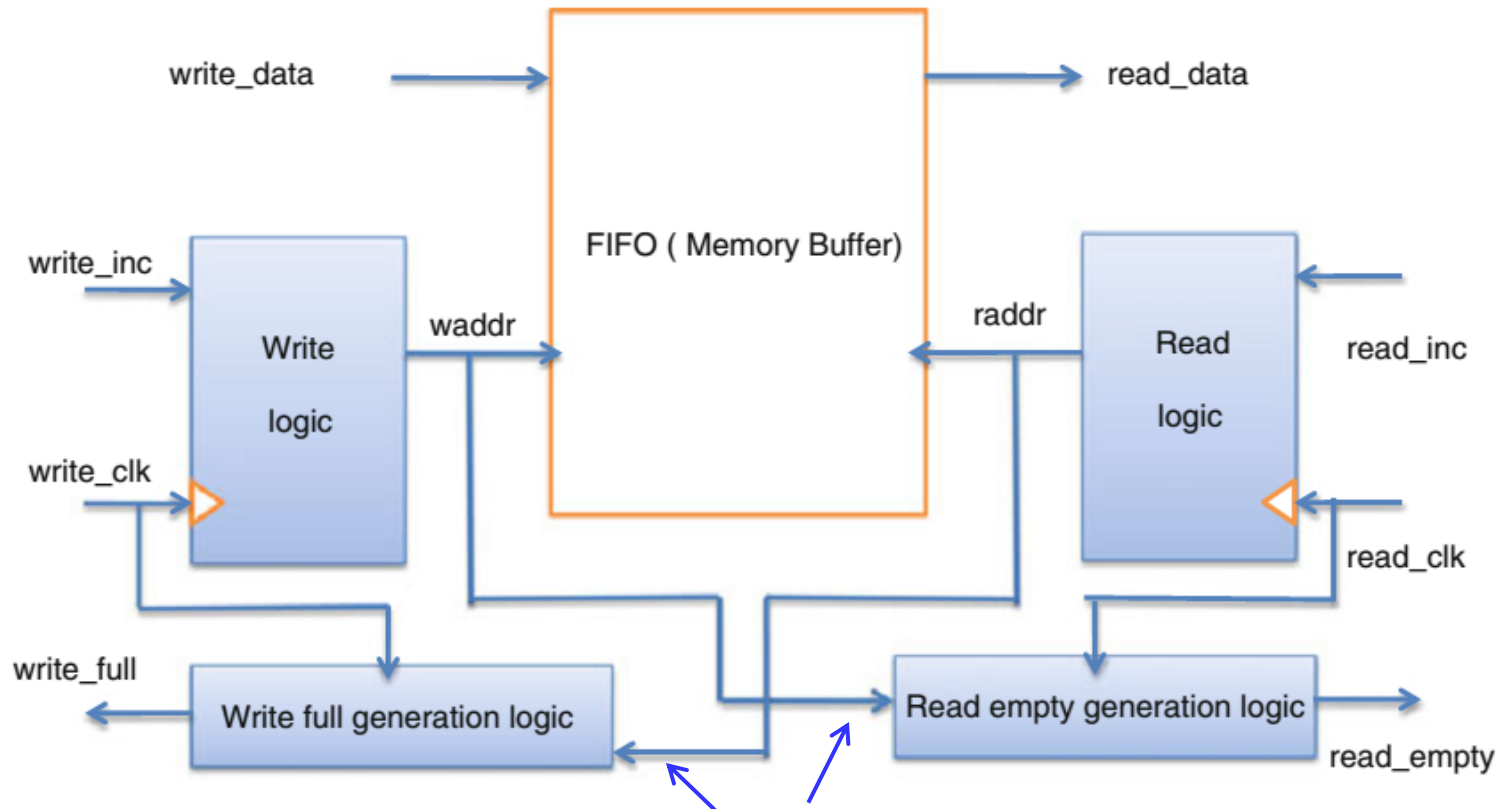
- **Handshaking Mechanism - synchronizer with feedback acknowledge**
 - One or more than one handshake signals are required while passing the data from one of the clock domains to another clock.



Signal Transfer across the Clock Boundary (continued)

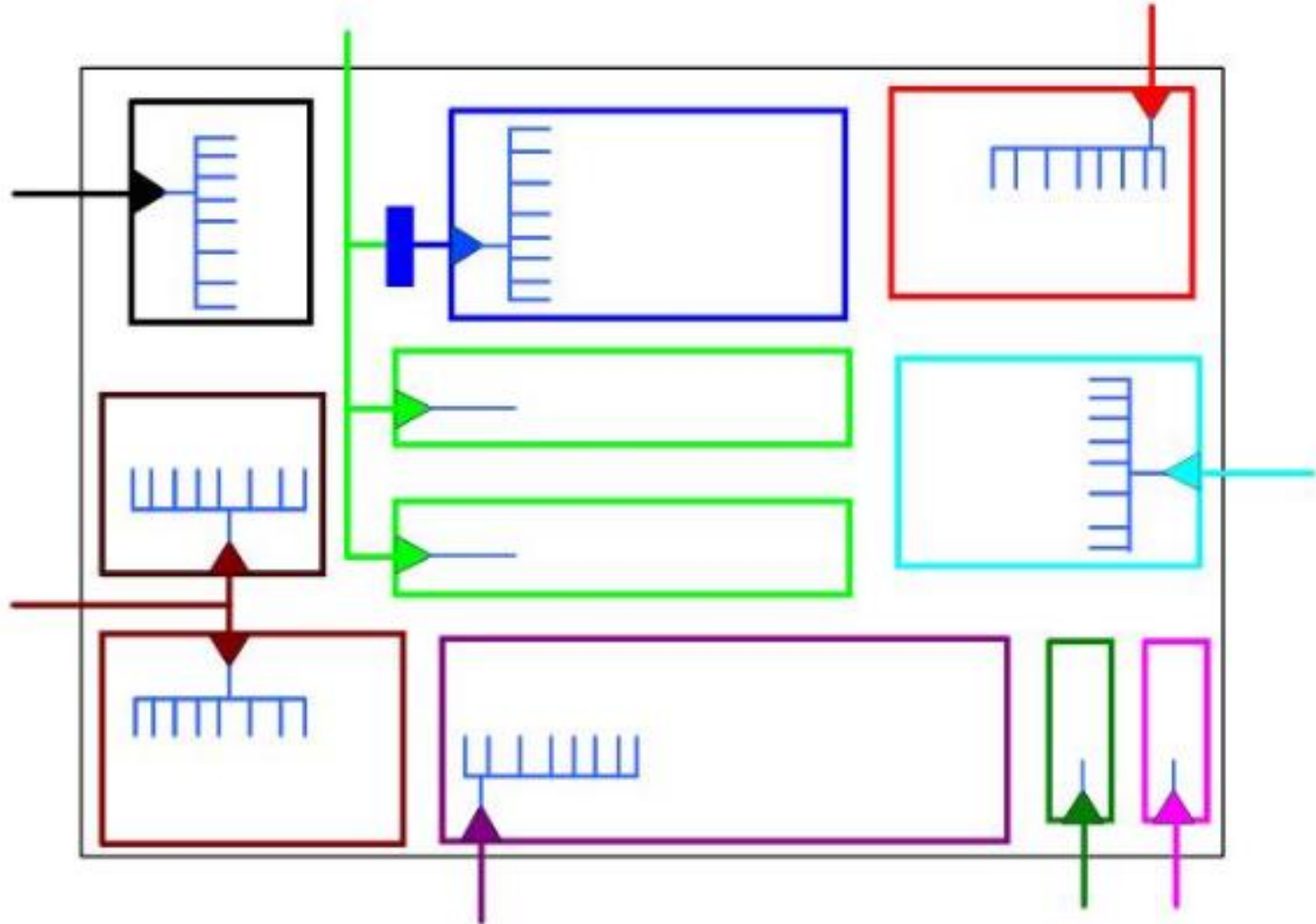
■ FIFO (First in First Out) Synchronizer

- FIFO memory buffers can be used as a synchronizer to pass the data between multiple clock domains.



moving address across clock domain

Digital System with Multiple Clock Domains

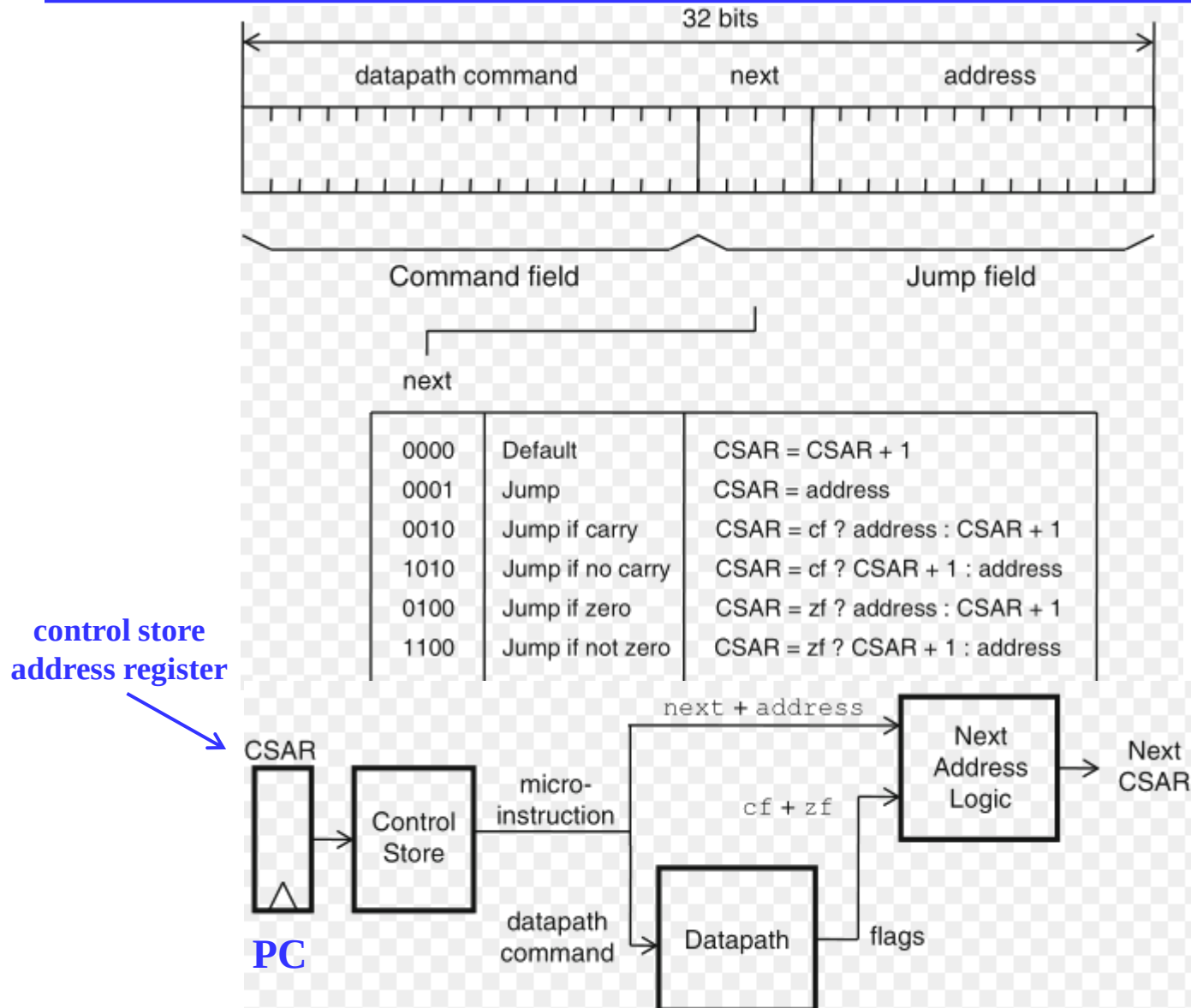


Appendix A: Signal Transfer across the Clock Boundary

Microprogrammed Control (continued)

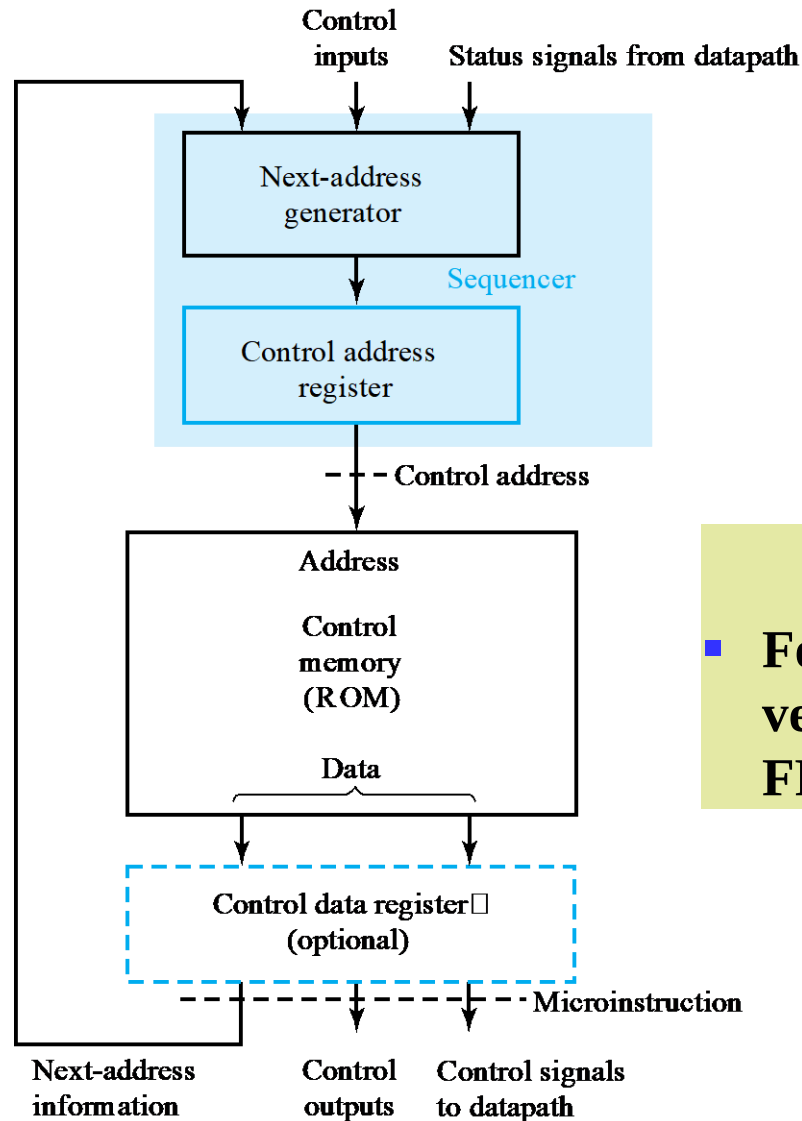
- *Microprogrammed Control* — a control unit with binary control values stored as words in memory.
- *Microinstructions* — words in the control memory.
- *Microprogram* — a sequence of microinstructions.
- *Control Memory* — RAM or ROM memory holding the microinstructions.
- *Writeable Control Memory* — RAM Memory into which microinstructions may be written

Microprogrammed Control (continued)



**Microinstruction
execution**

Microprogrammed Control (continued)



Reset Vector

- For example, the reset vector for x86 CPU is FFFF0h.